

ParkVision 智慧停车系统系统设计文档

1. 引言

1.1 编写目的

本文档是《ParkVision——基于边缘计算与多智能体协同的物理信息智慧停车系统》的系统设计规格说明书。

本文档的主要目的在于将前期的《项目开题报告》与《需求分析规格说明书》中的业务需求与系统愿景，转化为指导软件开发的工程化技术蓝图。本文档详细规定了 ParkVision 系统的总体架构、微服务组件划分、数据库模型、核心业务状态机流转逻辑以及前后端交互接口标准。

本文档的预期读者包括：

开发团队：作为前后端编码、AI 算法集成、数据库构建的标准依据，确保多模块开发的协同与统一。

测试团队：辅助理解系统内部流转逻辑，为制定系统集成测试、性能压测与异常回滚测试用例提供技术参考。

评审专家与指导老师：供项目答辩评审使用，清晰展现本项目的技术深度、架构合理性与工程落地能力。

1.2 项目背景与目标

项目背景：

随着城市化进程加速，城市核心区土地资源稀缺与机动车保有量激增的矛盾日益突出。为突破空间限制，工业界广泛引入了“AGV 托举式全自动封闭立体车库”。然而，现有的控制系统大多停留在基础的“资产管理”阶段，采用传统的“先进先出（FIFO）”线性调度逻辑。这种缺乏预见性的机械恒速调度，导致在早晚高峰或商场集中散场等突发流量下，系统极易陷入拥堵瘫痪；同时，复杂的室外或半开放物理环境也对视觉感知与全天候安全防线提出了极大挑战。

项目目标：

ParkVision 旨在构建一个基于**物理信息系统（CPS, Cyber-Physical Systems）**架构的

智慧停车控制中枢，通过“软件定义”的方式为庞大的底层 AGV 硬件集群注入预见性与灵活性。本系统的核心设计目标包括：

打破物理搬运瓶颈（化解排队死锁）：引入 AI 时序预测模型，通过“高峰波峰预测与前置调度（Pre-Dispatch）”，在非高峰期将高频出库车辆提前移至浅层缓存区，以空间换时间，大幅削减车主等待时长。

重塑柔性业务流转（提升交互体验）：通过引入分布式状态机引擎，打破传统车库“存了就不能动”的死板限制，实现诸如“Touch-and-Go 临时取物”、“加急抢占出库”以及“新能源满电腾桩”等复杂的现实业务流转。

构建全天候安全防线（软硬件协同）：采用“边缘计算+中心云”协同架构，在边缘端部署恶劣工况图像增强与非许可生命体识别模型，实现毫秒级物理急停，确保工业级绝对安全。

生成式 AI 深度赋能（数据驱动运营）：融合大语言模型（LLM），在 C 端实现自然语言意图解析助理，在 B 端实现基于 Text-to-SQL 的智能化报表聚合查询，降低系统使用门槛。

1.3 术语与缩写定义

为了便于文档阅读与避免歧义，特对本项目涉及的关键专业术语与缩写定义如下：

术语/缩写	英文全称	释义说明
CPS	Cyber-Physical Systems	物理信息系统。本项目的核心架构理念，指计算进程与物理过程紧密耦合的智能系统，实现对 AGV 集群的精准管控。
AGV	Automated Guided Vehicle	自动导引运输车。本系统中特指底盘较低、用于托举并自动搬运车辆的机器人硬件设备。
Pre-Dispatch	Pre-Dispatch Logic	前置调度（预调度）。本系统的核心算法亮点，指基于 AI 时序预测，提前将深库区车辆移至浅层缓存区的动作。
Touch-and-Go	-	临时取物模式。一种特殊的订单状态流转机制，允许车主短时取物后车辆自动回库，期间订单挂起且计费不中断。

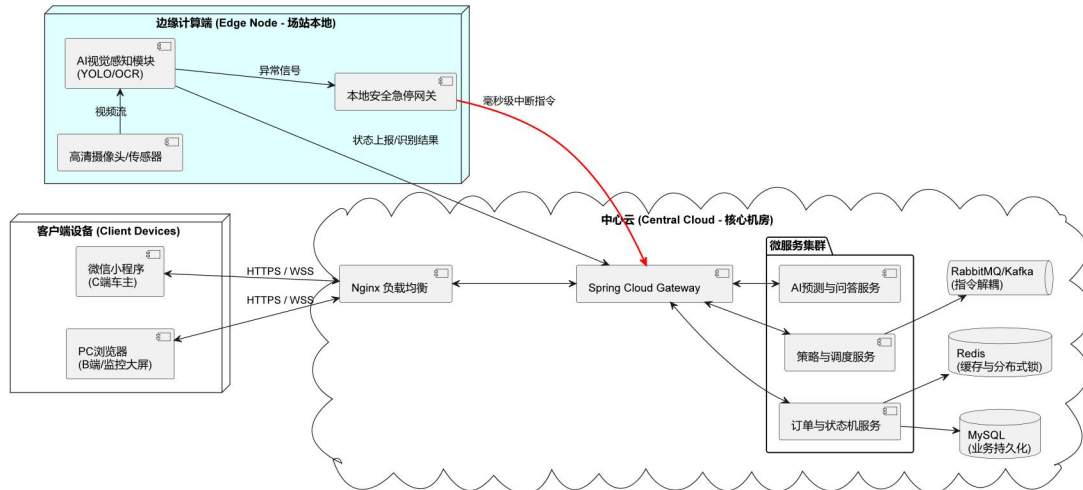
LLM / NLP	Large Language Model / Natural Language Processing	大语言模型 / 自然语言处理。用于车主语音助手意图识别与管理员 Text-to-SQL 报表生成的底层 AI 技术。
Text-to-SQL	-	一种 AI 交互范式，能够将管理员输入的自然语言（如“上周六营业额”）自动转换为数据库查询 SQL 并生成图表。
Redlock	Redis Distributed Lock	Redis 分布式锁算法。在系统调度核心中，用于确保高并发下 AGV 指令分配的唯一性与防死锁。
RBAC	Role-Based Access Control	基于角色的访问控制。后台管理系统中用于实现用户、角色、权限精细化管控的安全模型。
Digital Twin	-	数字孪生。本项目中指通过 WebSocket 将物理车库的 AGV 坐标与车位状态，以 1:1 高保真动态渲染到大屏 3D 网格模型中。

2. 总体架构设计

ParkVision 系统采用典型的“云-边-端”三位一体协同架构，旨在将复杂的业务逻辑计算与对时延要求极高的物理级安全控制进行物理层面的解耦，从而构建一个高可用、低延迟的物理信息系统（CPS）。

2.1 部署视图

本系统的部署架构从拓扑物理分布上划分为三个核心区域：客户端设备（Client Devices）、中心云核心机房（Central Cloud）以及部署在物理场站本地的边缘计算端（Edge Node）。



具体各部署节点的拓扑关系与职责说明如下：

2.1.1 客户端设备

客户端是系统与真实用户交互的触点，分布于公网环境中。为了保障数据传输的安全性
与实时性，客户端全部采用 **HTTPS** (加密超文本传输协议) 与 **WSS** (加密 **WebSocket** 通信协
议) 与云端网关进行通信。

微信小程序 (C 端车主)： 提供轻量级的移动端入口，满足车主在场内外发起存取车、
查看实时排队状态、唤起 **Touch-and-Go** 挂起以及 **AI** 语音问答等业务需求。

PC 浏览器 (B 端/监控大屏)： 运行于管理处或远端监控中心，提供高密度的管理后台面
板 (**Text-to-SQL** 报表) 以及基于 **WebGL/Canvas** 渲染的数字孪生 **3D** 监控大屏，依赖 **WSS**
长连接实现 **500ms** 级的状态刷新。

2.1.2 中心云 - 核心机房

中心云统管全系统的业务状态机、复杂策略计算及数据持久化，部署于具备高算力与高
可用保障的云服务器机房。

接入与流量控制层：

Nginx 负载均衡： 作为公网流量的第一道屏障，负责 **HTTPS** 证书卸载与反向代理，将
请求均匀分发至后端网关。

Spring Cloud Gateway： 作为微服务统一入口，承担路由分发、**RBAC** 鉴权拦截、全局限
流等核心安全职责。

微服务集群： 部署了系统的核心业务逻辑，包括 订单与状态机服务（维护生命周期防
死锁）、策略与调度服务（**VIP** 抢占、**Pre-Dispatch** 预调度）以及 **AI** 预测与问答服务（对接

大模型生成策略与报表）。

存储与中间件集群：

MySQL 8.0： 用于业务数据的结构化持久化存储（如账单、用户资产、拓扑关系）。

Redis 集群： 承担两项重任：一是缓存高频读取的 **AGV** 实时坐标；二是提供 **Redlock** 分布式锁，确保高并发发起订单时状态流转的强一致性。

RabbitMQ / Kafka： 消息中间件，用于将前端高并发的毫秒级业务请求与底层长达数分钟的机械动作指令进行异步解耦，防止系统线程阻塞。

2.1.3 边缘计算端 - 场站本地

边缘端部署在立体车库的物理场站本地工业控制柜或本地局域网内，其核心设计理念是“安全防线不下沉，控制闭环在本地”，以解决网络波动带来的物理碰撞风险。

高清摄像头/传感器： 部署于交接区与关键通道，实时采集视频流与物理环境数据。

AI 视觉感知模块 (YOLO/OCR)： 在本地服务器上直接对视频流进行深度学习推理。实现复杂工况下的车牌特征提取，以及针对交接区儿童、宠物等非许可生命体的越线检测。

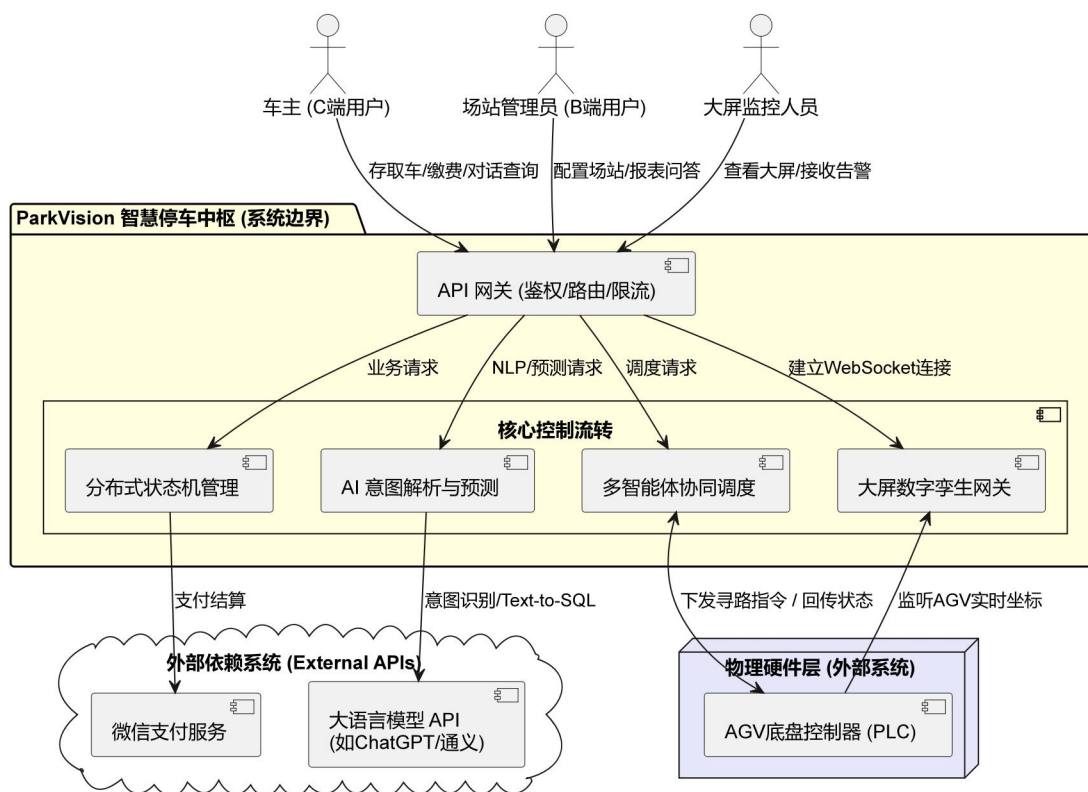
本地安全急停网关 (Safety Gateway)：

云端交互： 将正常的车牌识别结果与设备健康状态通过广域网定期上报给中心云。

本地熔断机制： 一旦 **AI** 视觉感知模块捕捉到“异常信号”（如人员误入作业区），该网关将直接绕过中心云的网络通信，向底层 **PLC** 硬件下发毫秒级中断指令（红线所示），强制触发物理急停，守住工业级安全底线。

2.2 逻辑架构图

逻辑架构旨在从数据流和功能边界的维度，阐述系统是如何接收不同角色的外部指令，并在内部完成“请求解析 -> 业务流转 -> 硬件执行”闭环的。



在 ParkVision 系统中，以“ParkVision 智慧停车中枢”为绝对的系统控制边界，将其逻辑组件拆解为以下三个核心交互维度：

2.2.1 多端角色的访问接入

位于系统边界最上方的是三类核心业务参与者，他们通过不同的协议与网关（API Gateway）建立连接：

车主 (C 端用户)： 产生高并发的业务请求，包括存取车指令、在线缴费结算，以及基于自然语言的对话式业务查询。

场站管理员 (B 端用户)： 产生低频但高权限的管理请求，包括场站拓扑结构的配置、策略规则定义，以及通过 Text-to-SQL 进行深度报表问答。

大屏监控人员： 发起 WebSocket 长连接请求，主要用于查看实时高保真大屏视图，并接收来自边缘端的异常告警推送。

API 网关拦截： 所有上述外部请求必须经过 API 网关，进行统一的身份鉴权、流量限流与动态路由，确保内部核心控制流的绝对安全。

2.2.2 核心控制流转中枢

进入系统边界内部后，请求被路由并转化为四类核心逻辑流转模块的协同作业：

分布式状态机管理：整个业务逻辑的心脏。负责维护订单与车辆从“入库-停妥-挂起-出库-结算”的全生命周期状态。它接收业务请求，并严格控制状态的单向扭转，防止并发冲突。

AI 意图解析与预测：作为系统的“大脑辅助”，它负责处理 **NLP** 预测请求。它将接收到的非结构化文本（如用户的模糊问答）转化为具体的动作指令，同时负责运行预调度（**Pre-Dispatch**）模型，推导波峰拥堵概率。

多智能体协同调度：承接状态机扭转产生的任务，负责将抽象的业务需求（如“将 **A12** 车位车辆移出”）转化为具体的多台 **AGV** 协同物理寻路逻辑与冲突控制调度。

大屏数字孪生网关：专门负责将后端的业务状态与坐标数据，高频打包并推送到前端，维持 **3D** 监控模型与物理场站的 **1:1** 状态映射。

2.2.3 系统外部依赖与物理交互

系统边界下方展示了 **ParkVision** 所依赖的外部软硬件生态，体现了软件定义硬件的架构思想：

外部软件依赖系统 (External APIs)：

微信支付服务：承接订单系统发起的支付结算请求，完成商业闭环。

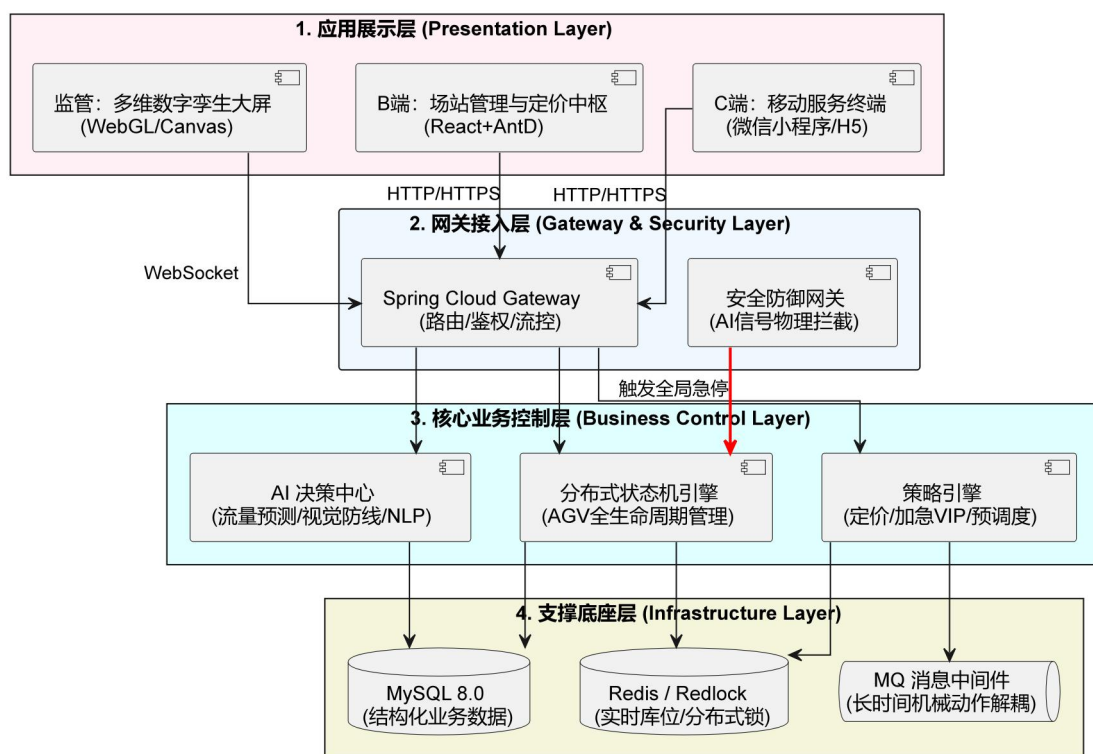
大语言模型 API (如 ChatGPT/通义)：系统将脱敏后的业务数据（**Prompt** 上下文）发送至外部大语言模型，利用其强大的语义理解能力实现用户意图识别与 **B 端 Text-to-SQL** 报表代码的生成。

物理硬件层 (外部硬件系统)：

AGV 底盘控制器 (PLC)：这是物理执行的终端。它不在本系统软件开发边界内，但系统通过标准化协议向其下发寻路指令数组，并实时监听其回传的运行状态与坐标数据。

2.3 系统架构图

为了保障代码的高内聚与低耦合，满足高并发业务需求及后续集群扩容的灵活性，**ParkVision** 系统严格遵循了四层分布式系统架构规范。各层级职责单一，层与层之间通过标准化协议进行交互。



系统的自上而下分为以下四个核心层级：

2.3.1 应用展示层

位于架构的最顶层，直接面向不同角色的终端用户，提供友好的跨平台人机交互界面。

C 端：移动服务终端： 基于微信小程序或 H5 技术构建，作为车主与物理车库交互的唯一入口，支持轻量级访问、LBS 定位导航及敏捷的业务操作交互。

B 端：场站管理与定价中枢： 基于 React 与 Ant Design 等前端框架构建，为运营者提供重度数据交互的 PC 端后台，实现拓扑图绘制、策略配置与复杂表单管理。

监管：多维数字孪生大屏： 采用 WebGL/Canvas 技术，负责承载高密度、高频刷新的坐标数据渲染，构建“所见即所得”的 3D 网格拟态监控中心。

2.3.2 网关接入层

这是系统的安全屏障与流量调度中枢，所有来自展示层的外部请求均需穿透此层方可触达核心业务。

Spring Cloud Gateway： 承担微服务架构中的标准网关职责。负责 HTTP/HTTPS 协议的统一接入、路由分发、RBAC（基于角色的访问控制）权限校验以及针对突发流量的限流削峰。

安全防御网关：这是一个结合边缘计算的特殊硬件级网关组件。负责接收并校验来自边缘端 AI 模型的物理检测信号。一旦触发“非许可生命体闯入”等告警，该网关将短路常规业务流，直接触发全局急停，构成物理信息系统（CPS）的最强安全防线。

2.3.3 核心业务控制层

这是 ParkVision 的“智慧大脑”，封装了所有的业务逻辑与算法推演。该层内部进一步划分为三大核心引擎：

分布式状态机引擎：作为系统流转的骨架，全权管理 AGV 及托管车辆的全生命周期。通过严密的状态转移约束，防范并发请求下的死锁或逻辑重入。

策略引擎：作为规则执行器，负责解析动态定价、计算 VIP 抢占插队的加权优先级，以及将抽象的业务请求转换为具体的 AGV 预调度任务与寻路指令。

AI 决策中心：作为系统的智能化大脑，集成了时序流量预测、边缘视觉防线监控与云端大模型 NLP（自然语言处理）意图解析能力，为策略引擎提供前瞻性的数据支撑与决策依据。

2.3.4 支撑底座层

位于架构的最底层，为上层高并发的业务逻辑与高频的状态流转提供坚实的存储与通讯基础设施。

MySQL 8.0：提供具备 ACID 特性的关系型数据库服务，负责所有结构化业务数据（如用户信息、订单记录、账单明细、场站空间拓扑配置）的持久化存储。

Redis / Redlock：提供内存级的高速缓存服务。一方面用于缓存高频刷新的车库实时库位与 AGV 坐标状态；另一方面提供基于 Redlock 算法的分布式锁机制，确保状态机引擎在极高并发环境下的数据强一致性。

MQ 消息中间件：采用异步消息队列机制，用于彻底解耦微秒级的前端软件指令与长达数分钟的物理机械搬运作业。消除因物理动作延迟导致的系统级线程阻塞，大幅提升吞吐量。

3. 核心微服务与组件设计

为实现系统架构图中规划的解耦目标，ParkVision 系统在核心业务控制层采用了微服务架构。本章将详细阐述各个核心组件的职责边界、内部结构以及支撑核心业务场景的动态交互时序。

3.1 组件设计概述

系统的业务逻辑被高度内聚并划分为五个平行的核心组件领域，它们通过轻量级的 RESTful API 与异步消息队列进行通信协同。

这五大核心组件的职责界定如下：

用户与权限 (User & Auth):

作为系统的基石，全权处理多端（C 端车主、B 端管理员、大屏监控员）的账号体系生命周期。负责管理车辆资产与用户账户的绑定关系，并基于 RBAC 模型实现后端接口级的细粒度权限控制与安全审计。

订单与状态机 (Order & State Machine):

系统的绝对核心中枢。负责统管一辆车从发起存车、入库、临时挂起、出库到最终结算的全生命周期。通过集成 Redis Redlock 分布式锁，彻底杜绝高并发压测下的订单状态流转乱序、死锁及超卖风险。

调度与策略引擎 (Dispatch & Policy Engine):

充当纯软件请求与物理硬件动作之间的“翻译层”。它负责承接状态机下发的抽象任务，结合时间窗、资源约束等规则，计算 AGV 任务队列的优先级权重（如 VIP 抢占插队），并将业务请求转化为底层的物理寻路规划指令。

AI 与数智决策 (AI & Decision Intelligence):

系统智能化能力的统管中心。向上桥接云端大语言模型，处理用户的自然语言指令（NLP）并自动生成统计报表（Text-to-SQL）；向下对接场站边缘端的视觉识别网络，同时在内部运行本地化的时序算法，执行流量预测模型，驱动预调度逻辑。

数字孪生与监控 (Digital Twin & Monitoring):

负责打破物理场站与数字空间的“黑盒”。通过维持高频数据通道，将物理车库中 AGV 的高频坐标位移与库位占用状态的瞬时变化，1:1 镜像同步推送至前端大屏或小程序，实现透明化、可干预的可视化监控闭环。

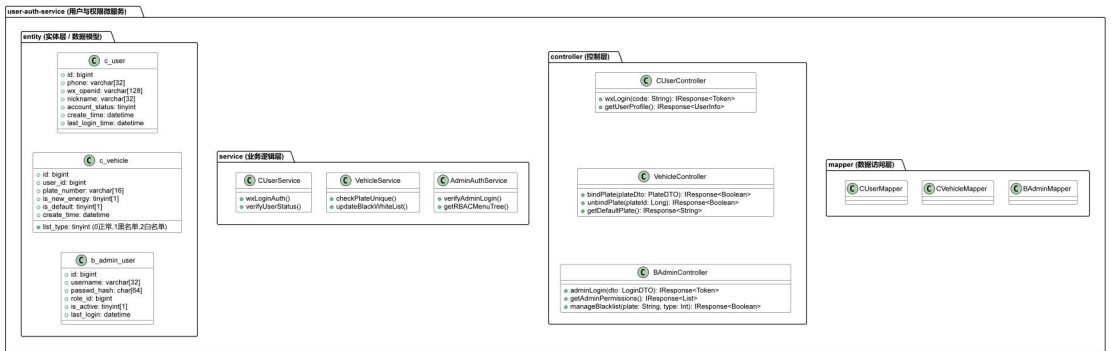
3.2 用户与权限微服务

用户与权限微服务是整个系统的基座防线。其核心设计目标在于：实现多端角色（C 端车主、B 端管理员、大屏监控员）的精细化权限治理，管理车辆资产的动态绑定，并在高并发登录请求下保障极高的响应速度与认证安全性。

3.2.1 组件内部结构与领域模型

本模块采用经典的 RBAC（Role-Based Access Control，基于角色的访问控制）模型构建

底层权限骨架，并在系统架构上实现了控制器层（Controller）与核心业务逻辑层（Service）的严格解耦。



数据实体层 (Entity):

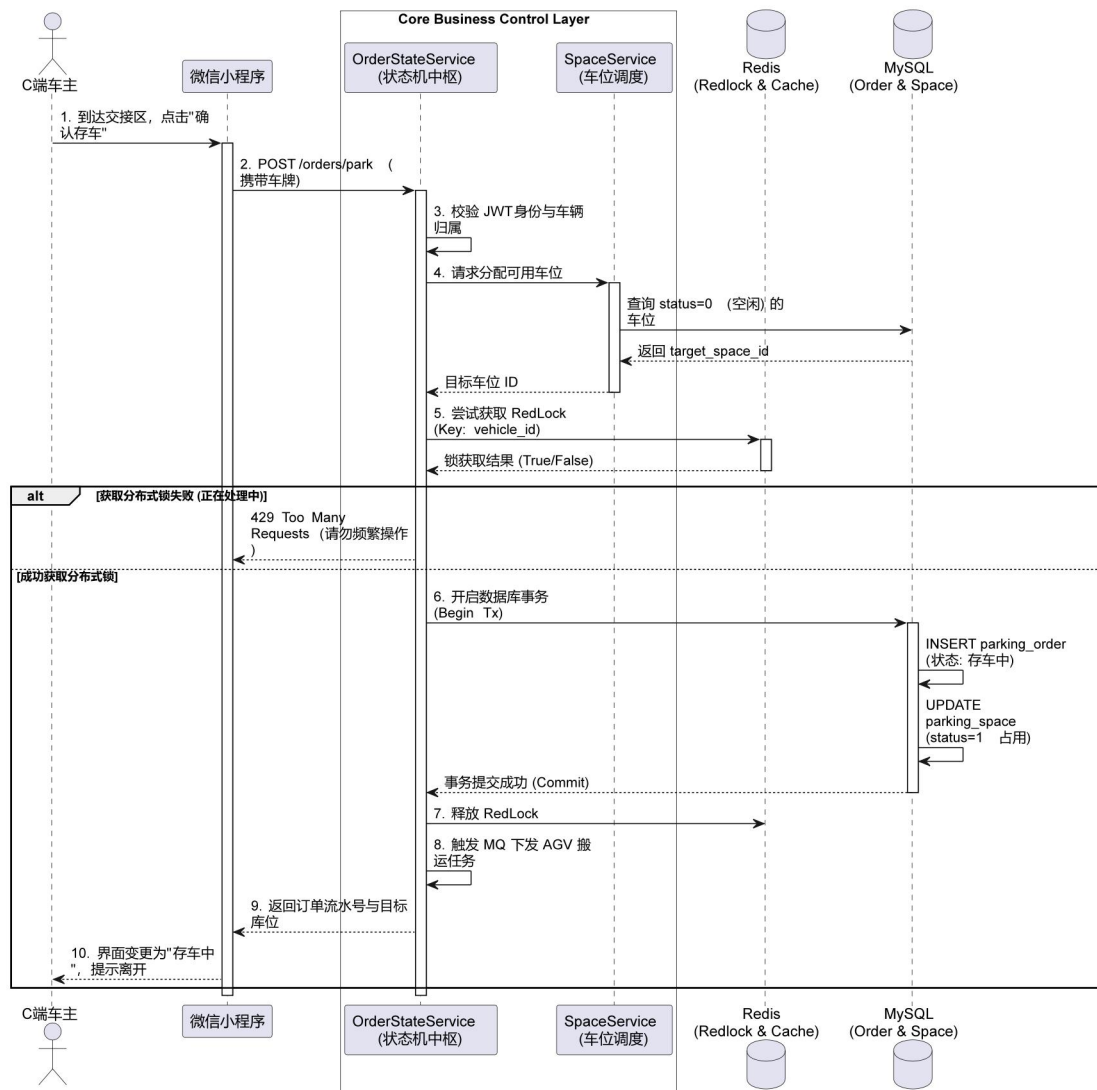
系统在实体层做了明确的物理隔离。**c_user**（车主）与**b_admin_user**（管理员）分别建立独立模型，防止了 B 端高危权限对 C 端业务库的污染可能。同时，系统深度定义了**c_vehicle**（车辆资产）实体，引入了诸如“能源类型”、“默认车辆”、“黑白名单属性”等扩展字段。这种设计使得一旦用户完成身份验证，系统能够根据其名下挂载的车辆属性，自动触发后端的差异化停车策略（例如：识别到新能源车牌，自动推荐带有充电桩的库位）。

业务逻辑解耦 (Service & Controller):

通过 **CUserService**、**AdminAuthService** 与 **VehicleService** 的分离，确保了面向移动端的轻量级校验（依赖微信体系）与面向后台的重度鉴权体系（依赖密码学 Hash 计算）互不干扰。

3.2.2 C 端车主登录授权时序

C 端车主登录追求的是极致的“零感交互”体验，系统依托微信小程序生态实现了静默授权机制，摒弃了传统的账号密码注册流程。



核心流转解析：

凭证获取：用户打开小程序，前端自动调用微信 API 获取临时登录凭证（WeChat Code）。

安全置换：user-auth-service 接收该凭证，并在服务端进行非空与合法性校验后，向微信开放平台服务器发起 HTTP 请求，安全置换出用户的唯一标识（openid）。

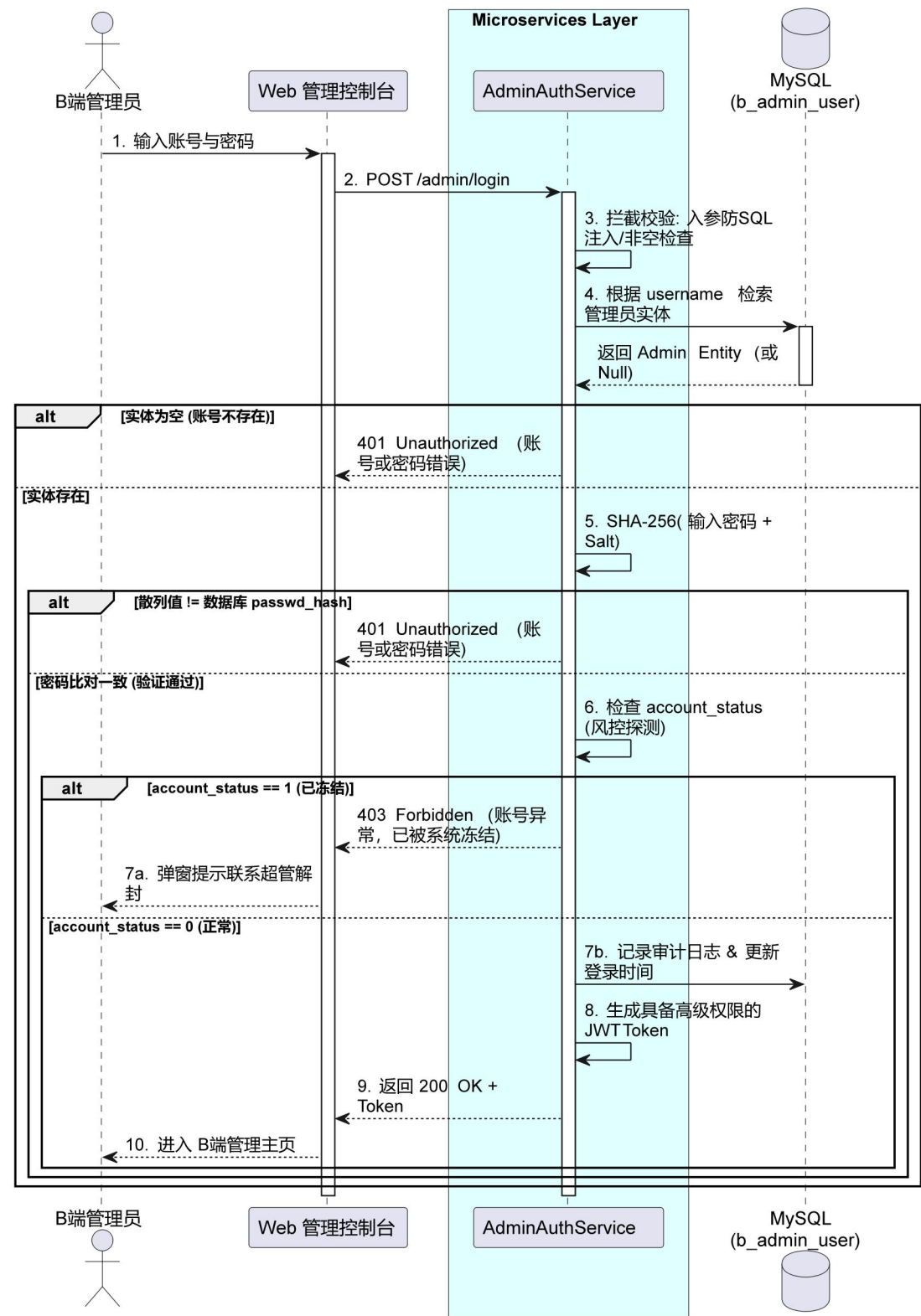
无缝注册与签发：系统根据 openid 检索 c_user 表。若为新用户，则静默完成用户实体的创建；随后，利用安全密钥生成包含用户身份特征与过期时间的 JWT (JSON Web Token)。

状态返回：最终将 Token 与脱敏后的用户信息下发至客户端。后续的所有存取车请求均需在 HTTP Header 中携带此 Token，以实现无状态的安全鉴权。

3.2.3 B 端管理员风控拦截时序

相较于 C 端，B 端管理控制台拥有修改计费规则、操控场站设备等极高权限，因此其认

证时序侧重于严密的安全校验与异常风控拦截。



核心防线解析:

凭证保护: 管理员输入的密码在前端经过初步处理后, 通过 HTTPS 密文通道提交至后

端。

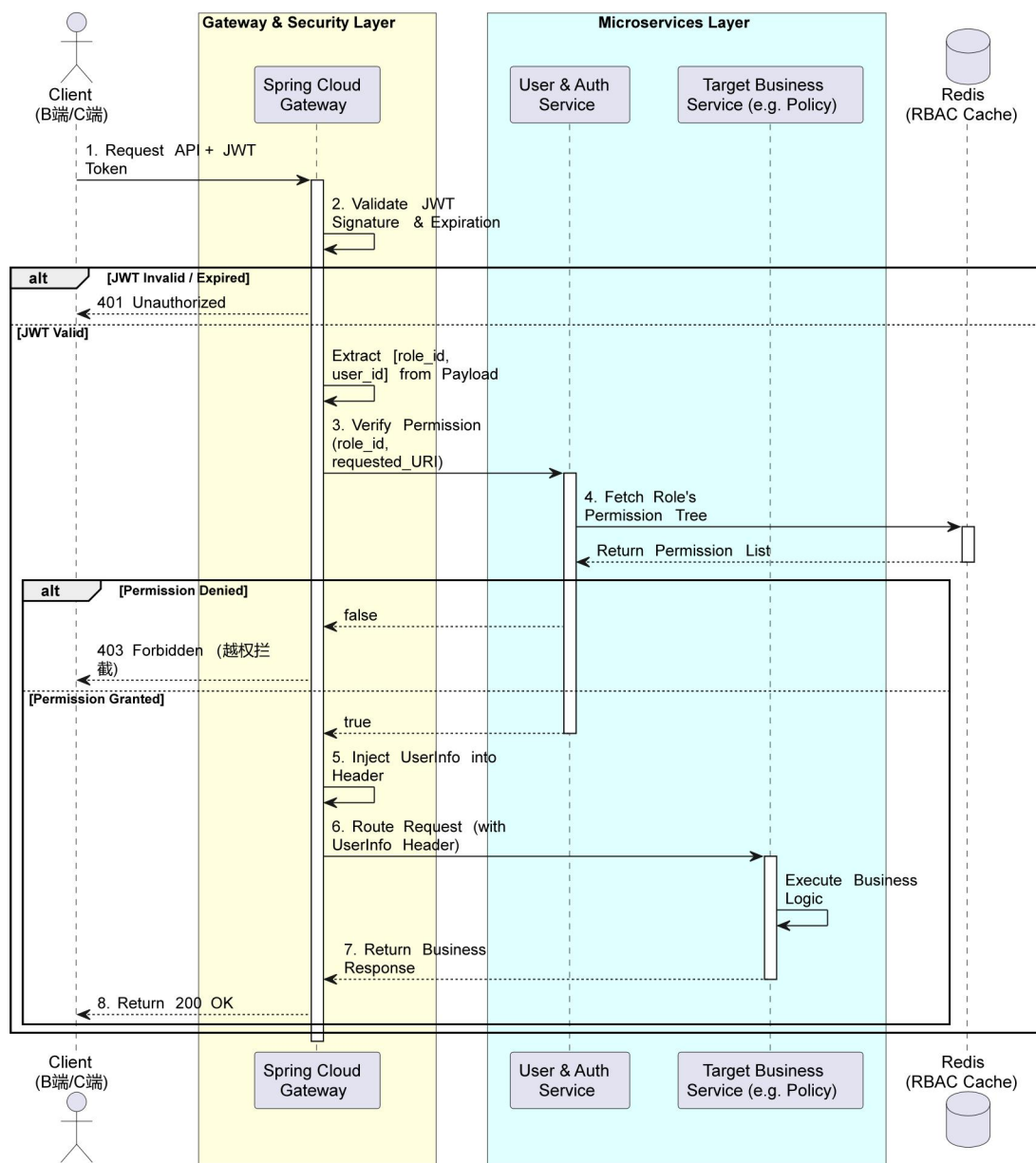
安全散列校验： 后端接收到凭证后，绝不进行明文比对。系统根据用户名提取 `b_admin_user` 实体，并对传入的密码执行加盐（Salt）散列计算（如 SHA-256 算法），将计算结果与数据库存储的 `passwd_hash` 散列值进行比对，确保数据库即便泄露也能保证密码安全。

风控状态机核验： 在密码验证通过（Yes）之后，时序图揭示了一个至关重要的安全环节——账号状态核验。系统必须校验 `account_status` 字段（例如 `is_active == 0` 代表正常，1 代表冻结）。

异常拦截与留痕： 一旦系统检测到该管理账号因异常操作或离职已被冻结（`account_status != 0`），认证流程将在此处强行阻断。系统会生成安全告警日志，并向前端返回明确的 `Account Forbidden` 拒绝信息，彻底阻断非法侵入。

3.2.4 网关级统一鉴权与 RBAC 拦截机制

在微服务架构中，如果每个子服务都独立处理权限校验，将导致严重的逻辑冗余与性能损耗。因此，ParkVision 将 RBAC 的鉴权动作“前置”到了 Spring Cloud Gateway 网关层。配合 JWT 的无状态特性，实现了高并发下的毫秒级权限拦截。



核心逻辑解析：

统一拦截： 客户端请求敏感业务接口（如修改计费规则）时，在 HTTP Header 中携带 JWT Token。API 网关作为第一道防线，拦截所有进站流量。

Token 解析与防伪： 网关首先对 JWT 签名进行非对称密码学验证，校验 Token 是否被篡改或过期。此过程完全在内存中完成，无需查询数据库，极大降低了 IO 消耗。

RBAC 动态权限断言： Token 校验合法后，网关解析出当前用户的 role_id，并向 Auth Service 发起内部高速 RPC 调用（或查询 Redis 缓存的权限树），验证该角色是否具备当前请求 URI 的访问权限（Permission）。

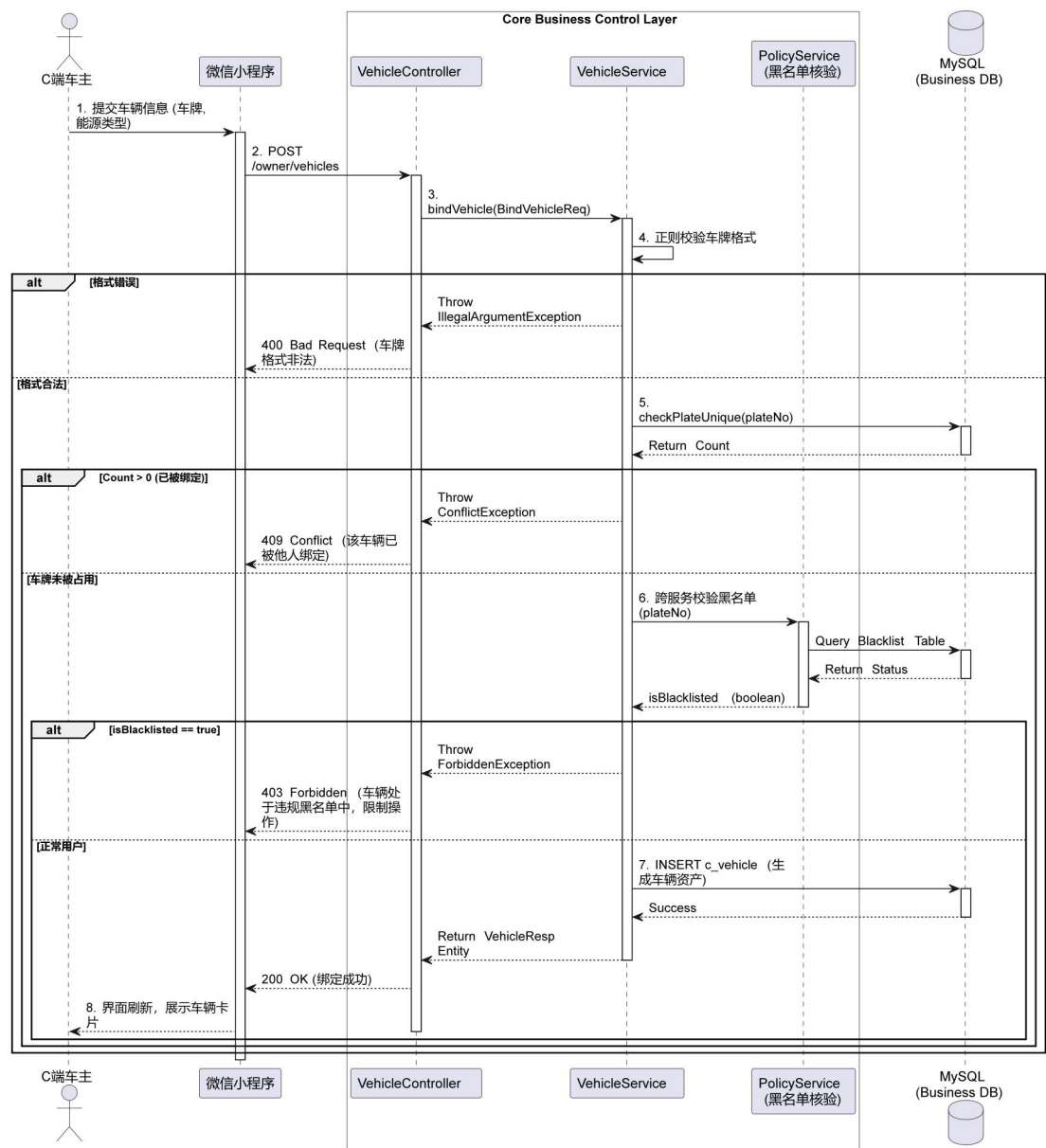
决策分发：

若鉴权通过，网关将解析出的用户明文信息（如 `user_id`）重新注入请求 Header，路由透传至下游的具体业务微服务。下游服务完全信任网关传递的信息，专注执行业务逻辑。

若鉴权失败，网关在最外层直接熔断，返回 `HTTP 403 Forbidden`，有效抵御越权攻击。

3.2.5 核心资产绑定与黑名单校验时序

车辆（Vehicle）是本系统中最核心的物理资产，所有计费、调度均围绕车牌展开。在用户绑定车牌的环节，系统设计了严密的风控逻辑，确保“黑名单用户无法入场”以及“资产唯一性防冲突”。



核心逻辑解析：

格式与唯一性初筛：车主提交车牌号（如“浙 A·12345”）与能源类型后，VehicleService 首

先通过正则表达式校验车牌合法性。随后执行严格的排他性检查（`checkPlateUnique`），防止同一车牌被多账号恶意绑定导致的计费主体混乱。

黑名单前置拦截：这是保障场站营收安全的关键。系统在落库前，会调用黑白名单检测接口（`checkBlacklist`）。如果该车牌因“历史严重逃单”或“多次违停”被管理员录入全局黑名单，系统将直接抛出异常，拒绝绑定，从源头上切断非法入场可能。

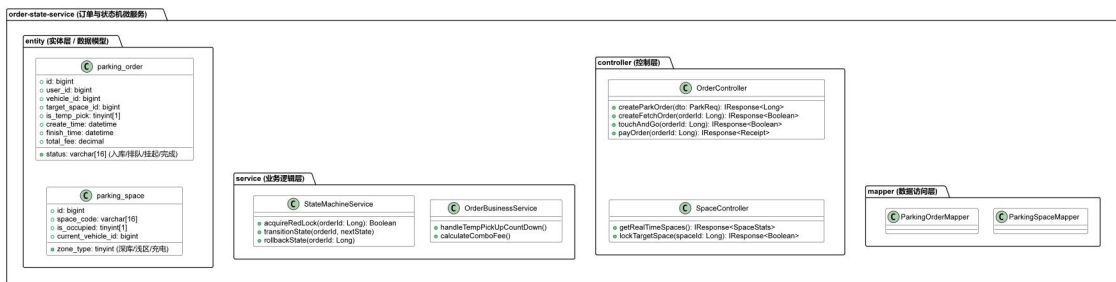
能源标签化处理：验证通过后，系统持久化车辆实体，并根据用户选择的“新能源”标签，同步更新系统底层的调度权重策略配置。这使得该车辆在后续入场时，能够被分布式状态机自动识别，并分配至带有充电桩的专属分区。

3.3 订单与状态机微服务

订单与状态机微服务是 ParkVision 系统的绝对流转心脏。它横跨了软件数据流与物理机械流，充当了“数字世界”与“物理车库”之间的桥梁。其核心设计挑战在于：在面对商场散场、早晚高峰等极高并发请求时，如何绝对保证“物理空间不冲突”、“底层机械不死锁”以及“计费账单不超卖”。

3.3.1 组件内部结构与状态机模型

该模块在底层抛弃了简单的 CRUD（增删改查）逻辑，全面引入了有限状态机设计模式，严格规范了订单流转的生命周期。

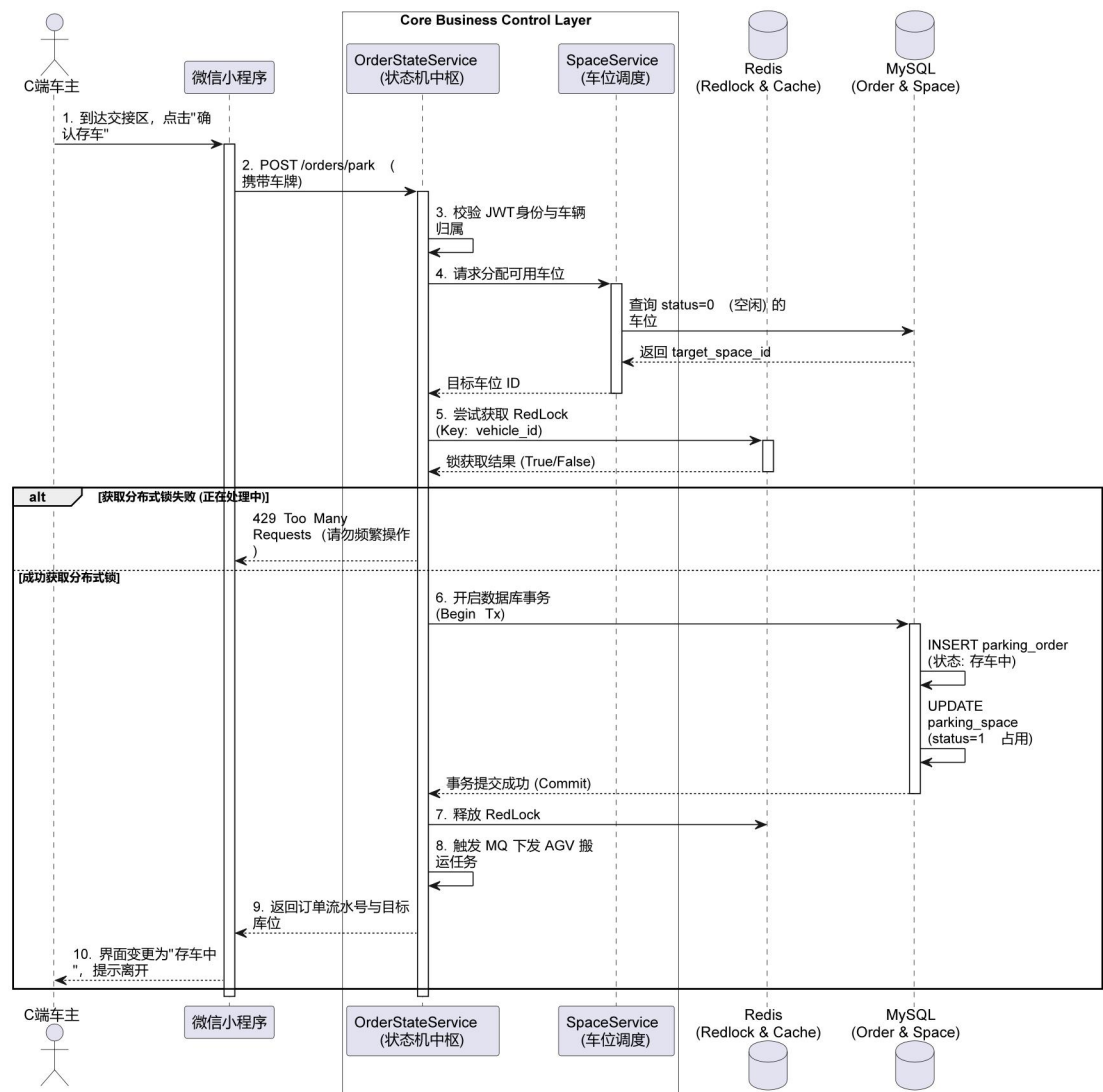


数据模型基石：模块以 `parking_order`（停车订单）为主轴，深度绑定了物理层面的 `parking_space`（车位资产）。其中，订单的 `status` 字段受到状态机的严密接管，定义了诸如 `Parking`（入库中）、`Parked`（已停妥）、`Pending`（取物挂起）、`Retrieving`（出库中）、`Finished`（已完成）等单向扭转节点。

防并发死锁设计：在 `StateMachineService` 中，深度集成了 Redis Redlock 分布式锁机制。任何引起状态变迁的操作，必须先在 Redis 层级抢占基于 `vehicle_id` 或 `space_id` 的分布式锁，从最底层杜绝了并发幻读导致的两车相撞或同一辆车被同时派发存取指令的风险。

3.3.2 发起存车防冲突时序

传统的立体车库在多用户同时扫码存车时，极易发生“车位重分配”的物理冲突。本系统通过“分布式锁+数据库事务+异步 MQ 消息”的三段式架构，彻底化解了这一危机。



核心流转逻辑更新解析:

可用空间嗅探: 用户在交接区确认存车后, 系统首先从数据库中嗅探当前状态为 status=0 (空闲) 的最佳目标车位。

Redlock 互斥抢占: 确定目标后, 系统立即向 Redis 申请基于 vehicle_id 的分布式互斥锁。如果由于网络抖动导致用户狂点按钮, 系统在此处即可拦截并发请求, 优雅降级返回 429 Too Many Requests。

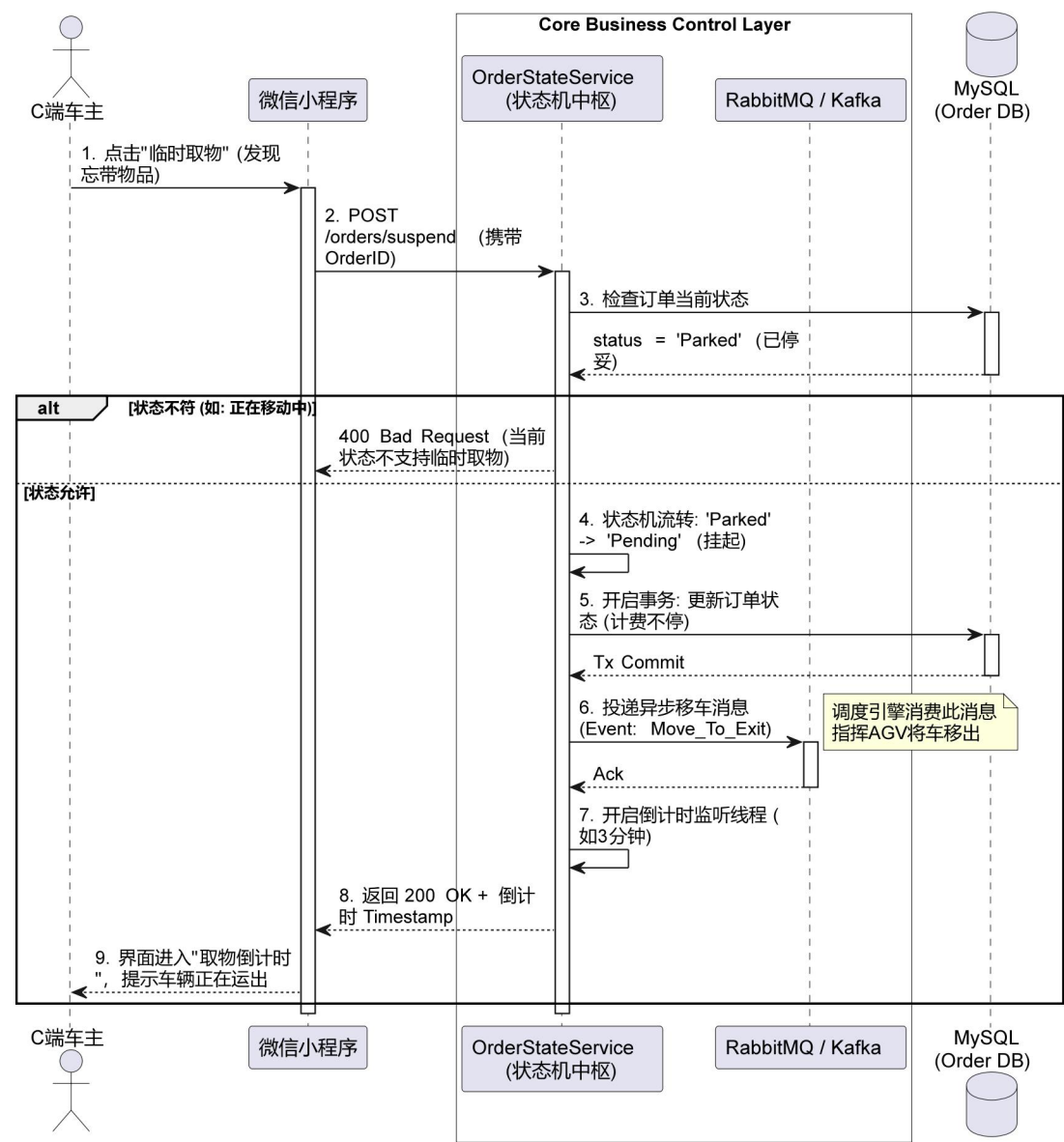
强一致性事务提交: 获取锁后, 开启 MySQL 本地事务。同时将生成订单 (存车中) 与锁定物理车位 (更新为占用状态) 打包在一个 Tx(Transaction) 内提交, 确保数字资产的一致性。

异步动作解耦: 数据库落盘后即刻释放锁, 并将长达数分钟的“AGV 搬运物理指令”通

过 RabbitMQ / Kafka 投递给下游的调度服务。前端快速响应并提示用户离开交接区，极大提升了软件侧的响应吞吐量。

3.3.3 Touch-and-Go 临时取物挂起时序

这是本系统打破传统自动化车库僵化体验的核心业务流。车主遗忘物品无需结算重停，系统通过状态机分支与物理调度协同，实现“计费不停、即取即走”。



核心流转逻辑更新解析:

状态机安全校验: 当系统收到 `suspend` 挂起请求时, 必须严格校验当前订单状态是否为 `Parked` (已停妥在库内)。若车辆正在物理移动中, 拒绝请求, 防止机械执行指令发生混乱。

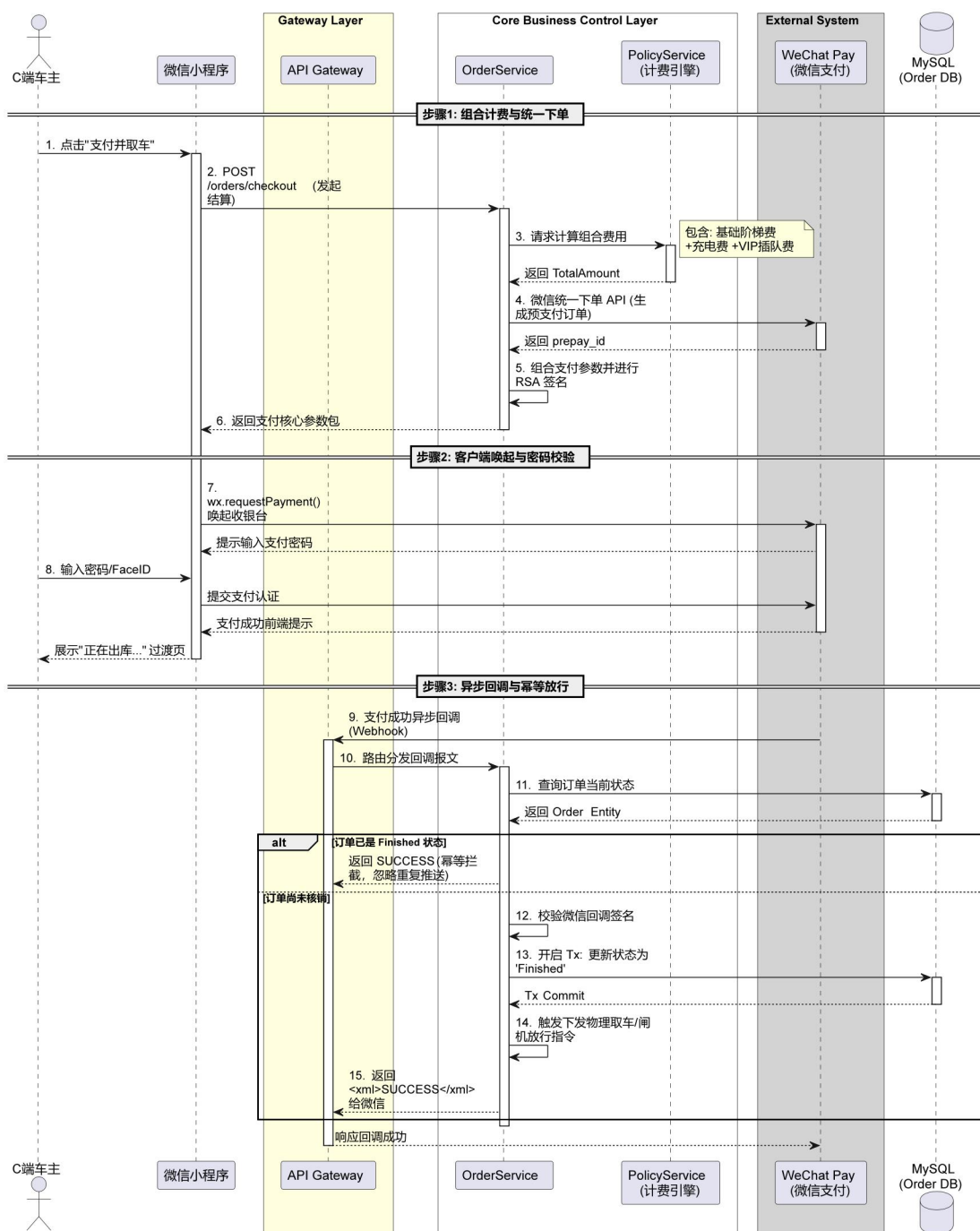
状态跳转与异步驱动: 校验通过后, 状态机将订单扭转为特殊的 `Pending` 状态, 并将其

持久化。随后立刻向 MQ 发送 Move_To_Exit（移至出口）的异步事件。

调度层消费与倒计时：底层调度引擎消费该消息，驱动 AGV 将车辆平移至交接区；同时业务层启动挂起倒计时（如 3 分钟），并在前端展示。在此期间，底层计费服务不受任何影响，实现了柔性的人机交互体验。

3.3.4 复杂计费结算与支付回调时序

当车主完成取车或申请 VIP 抢占出库时，系统进入最为复杂的账单结算环节。该环节需要横跨订单、策略、微信支付三大子系统，并处理支付过程中的异步一致性问题。



核心逻辑解析：

策略引擎组合计价：计费不再是简单的时长相乘。**OrderService** 需调用 **PolicyService**，将基础阶梯停车费、**Touch-and-Go** 产生的额外服务费、新能源充电电费以及 **VIP** 插队费进行多维度折算，生成最终的组合账单。

统一下单与参数签名：后端调用微信支付平台 **API**，获取 **prepay_id** 并生成带 **RSA** 签名的支付参数返回给小程序，唤起微信原生支付收银台。

异步回调幂等性处理：用户输入密码支付成功后，微信服务器会向我们的网关发起异步回调通知。为防止微信重复推送导致的“多次销账”，状态机在处理回调时加入了幂等性校验：仅当订单状态不为 **Finished** 时才执行支付成功逻辑。

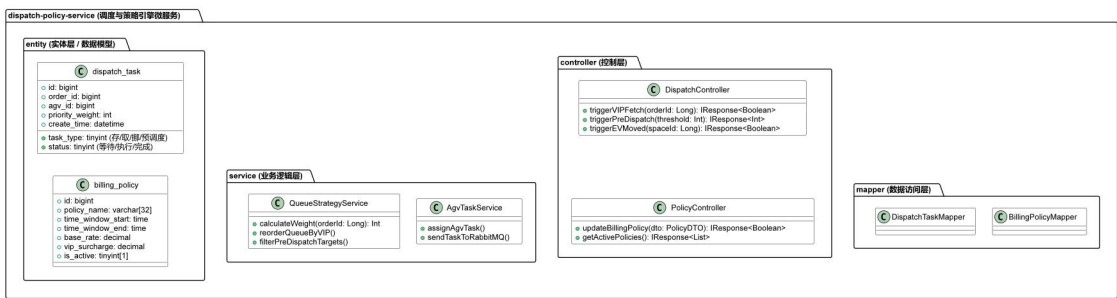
触发物理放行：账单结清、状态变更为 **Finished** 后，系统方可向物理闸机或 **AGV** 发送最终的释放指令，形成完整的“业务-资金-物理”闭环。

3.4 调度与策略微服务

如果说状态机是系统的核心，那么调度与策略微服务就是系统的**“小脑”与“翻译层”**。它的核心职责是接管来自状态机引擎的抽象业务请求（如“取车”），并将其翻译为底层物理硬件（**AGV** 集群）能够理解的机械寻路指令。同时，它肩负着处理所有与时间、资源冲突相关的策略规则调度。

3.4.1 组件内部结构与策略流转架构

该模块在架构设计上彻底剥离了“业务流转”与“物理执行”，通过构建独立的任务实体模型和异步消息通道，实现了软硬件解耦。



执行实体剥离：系统引入了 **dispatch_task** (调度任务) 实体。当用户的订单 (**parking_order**) 进入出库/移库状态时，本服务会为其生成一条对应的 **dispatch_task**。该任务脱离了用户属性，单纯包含物理维度的 **agv_id**、起点坐标、终点坐标与执行权重。

策略与队列引擎：模块内部由两个核心 **Service** 驱动：

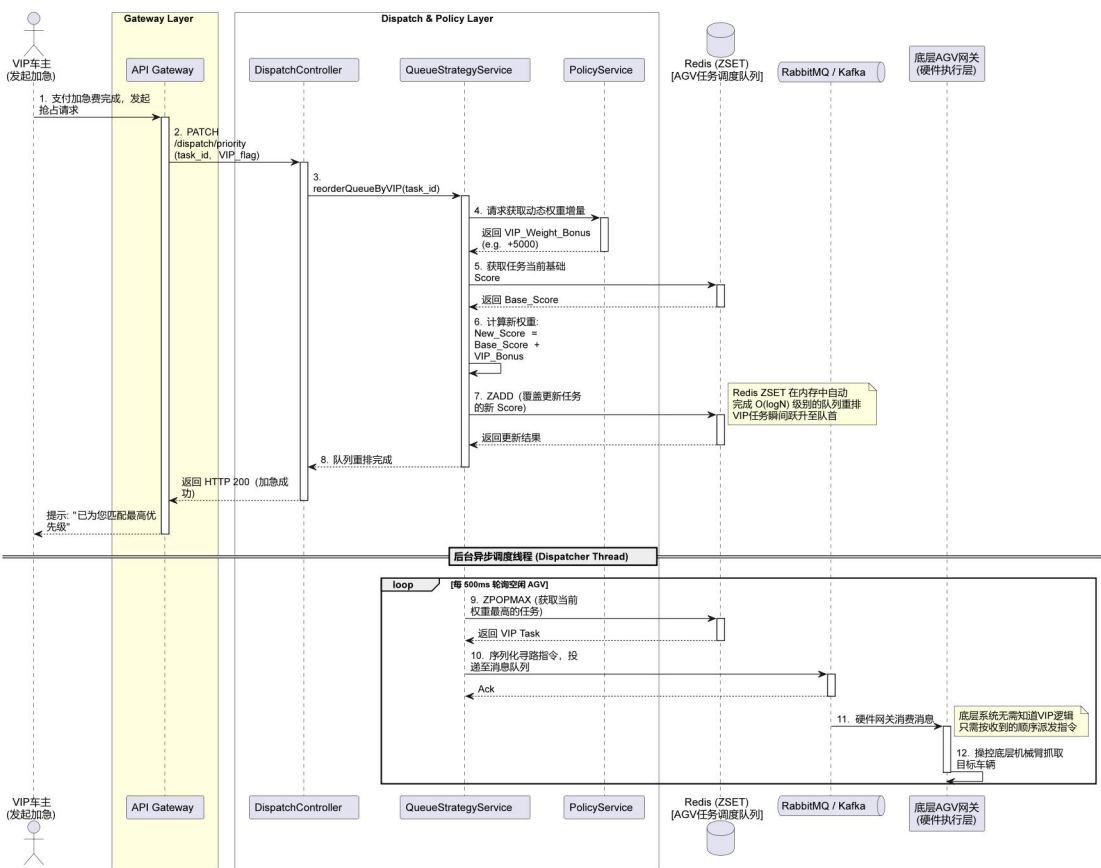
PolicyController / BillingPolicy: 负责读取后台配置的计费与优先级规则。

QueueStrategyService: 核心队列调度器。它维护着一个基于实时权重（Weight）排序的动态任务队列，负责计算每个任务的优先级分值，并决定哪个任务优先出队执行。

3.4.2 加急出库(VIP 插队)与动态权重重排时序

传统的先进先出（FIFO）队列在面临紧急突发状况时显得极为死板。本系统通过集成 Redis Sorted Set (ZSET, 有序集合) 数据结构，实现了一种支持 Weight Re-ranking（权重实时重算）的抢占式调度算法。

本节以“车主支付加急费，触发 VIP 优先取车”的场景为例，展示纯软件请求是如何打破物理排队顺序的。



核心流转逻辑解析:

触发抢占接口: 车主在小程序端发起加急申请（RESTful API 路径规划为 PATCH /api/v1/dispatch/priority）。

权重重算算法 (Weight Calculation): QueueStrategyService 接收到指令后，拦截该任务。系统根据策略引擎配置的公式重新计算其权重分值。例如：基础排队分数为 100，每等待 1 分钟加 1 分；触发 VIP 加急则一次性附加 5000 分的绝对压制权重。

Redis ZSET 动态重排：服务调用 Redis 的 ZADD 指令，利用更新后的权重分值覆盖原任务的 Score。Redis 自动在内存中完成 $O(\log(N))$ 级别的高性能队列重排，VIP 任务瞬间跃升至队列顶部。

出队与异步分发：调度线程通过 ZPOPMIN/ZPOPMAX 弹出当前队列中权重最高的任务，封装成标准机械指令报文，投递至 RabbitMQ 交换机。

硬件网关消费：底层 AGV 硬件网关无脑消费 MQ 中的消息，直接指挥空闲 AGV 前往该 VIP 车辆所在的深层库位进行搬运，完美实现物理世界的“插队”。

柔性补偿机制：为了保障用户体验，调度引擎在重排队列后，会计算被“挤占”导致预计等待时间延长的普通用户，并通过 WebSocket 向他们推送“调度延误补偿预警”，体现人性化服务。

3.5 AI 与数智决策微服务

在传统的仓储停车系统中，控制中枢往往是被动响应前端指令的“盲盒”。ParkVision 系统的突破性在于引入了 AI 与数智决策微服务，使其成为具备“感知力”与“预见性”的智慧大脑。该模块统筹管理大语言模型（LLM）、时序预测算法与边缘端机器视觉网络，为系统提供柔性交互与超前调度的决策依据。

3.5.1 组件内部结构与数智融合架构

该模块在架构上扮演着“承上启下”的角色：向上对接云端公有 AI 算力，向下挂接本地化的小参数预测模型与边缘视觉防线。



LLM 集成服务 (LLMIntegrationService):

负责处理自然语言交互。在 C 端，解析车主语音或文本中的模糊意图（如“充完电帮我挪一下”），将其转化为结构化的 Intent API 调用；在 B 端，利用大模型的 Text-to-SQL 能力，将管理员的自然语言提问（如“对比过去七天 VIP 插队收益”）动态编译为复杂 SQL，穿透数据库并生成前端图表。

时序预测引擎 (TimeSeriesPredictService):

运行于后台的轻量级预测算法。负责分析历史订单留存率、时间序列（早晚高峰、节假日

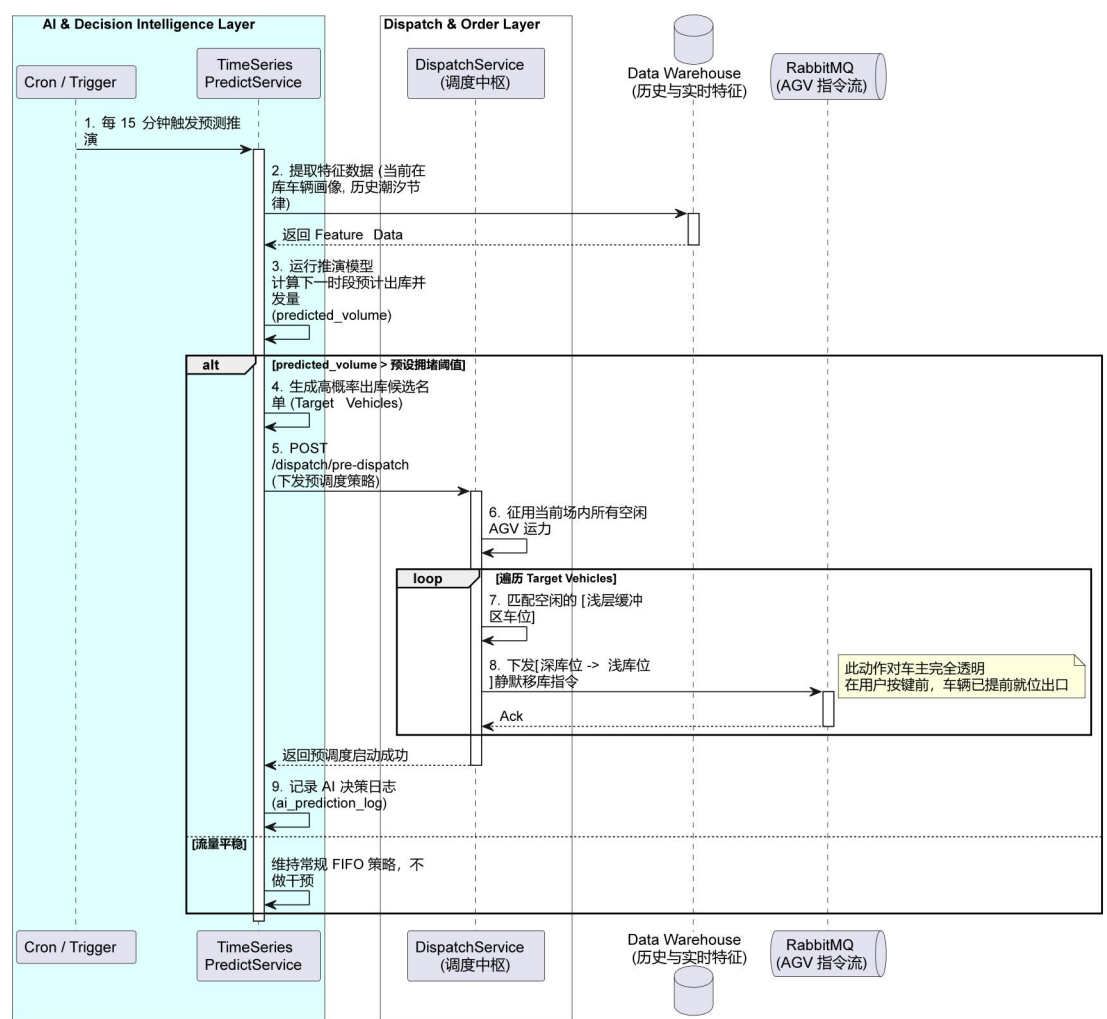
日) 以及外部交通热力特征, 输出未来 1-2 小时的场站流量波峰预判。

边缘视觉中继服务 (EdgeVisionService):

直接建立与场站物理边缘节点 (Edge Nodes) 的通信链路, 接收基于 YOLO 等目标检测算法上报的异常状态日志与高危拦截信号。

3.5.2 高峰波峰预测与 Pre-Dispatch 预调度时序

这是本项目解决“自动化机械恒速”与“瞬时突发流量”这一供需矛盾的核心杀手锏算法。系统通过“以空间换时间”的深浅库位翻转逻辑 (Deep/Shallow Swapping), 彻底打破了传统的 FIFO 效率瓶颈。



核心流转逻辑解析:

定时特征提取: AI 引擎内部的 Cron 定时任务 (例如每 15 分钟) 触发, 从数据仓库提取当前在库车辆画像及历史潮汐数据特征。

算法推演与阈值比对: 时序预测模型 (如基于 LSTM 或时序回归算法) 推演出下一时

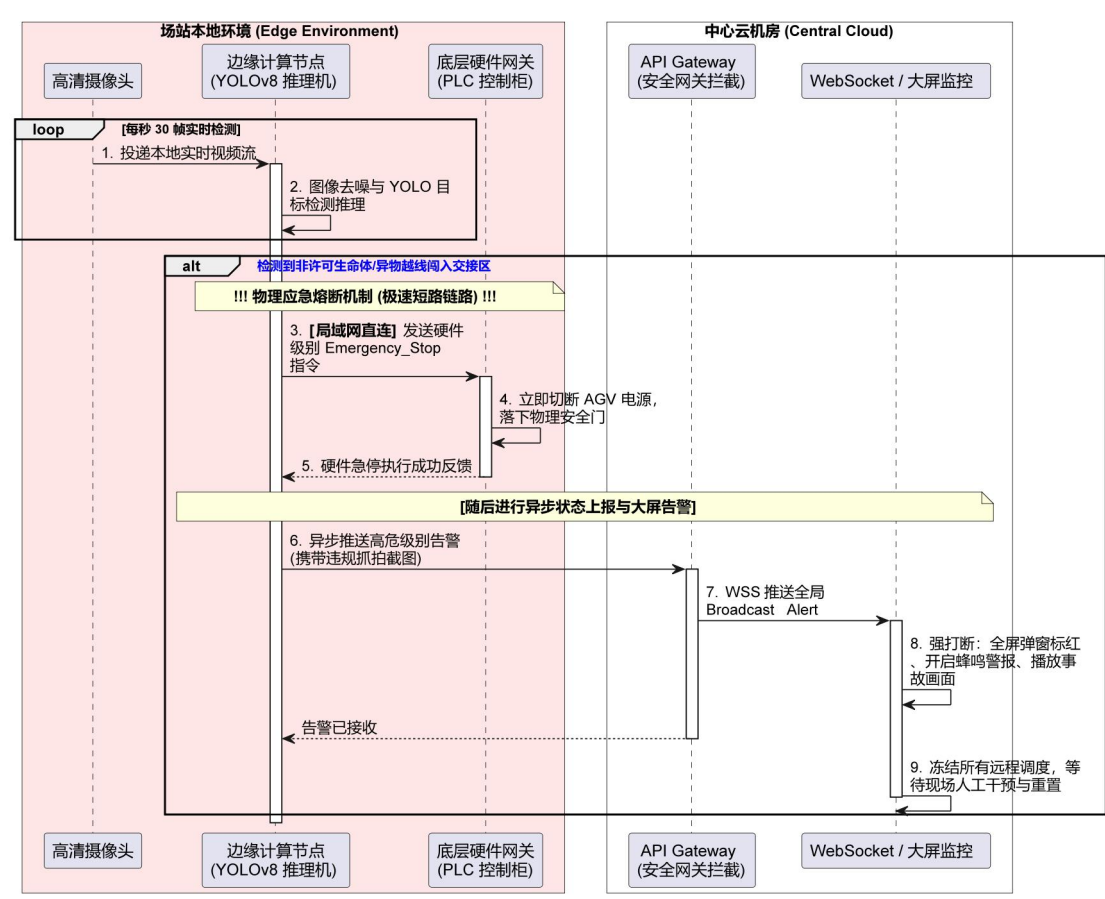
段的预计出库并发量。当 `predicted_volume` 大于系统设定的承载红线时，触发高级别的防拥堵警报。

闲置运力剥夺：系统立即向调度引擎发送指令，接管当前所有处于空闲待命状态的 AGV 集群。

深浅库位翻转 (物理动作)：系统根据出库概率模型，锁定一批最可能离场的车辆，向闲置 AGV 下发前置移库指令。在车主真正发起取车请求之前，AGV 已提前将车辆从耗时 3-5 分钟的“深层存储区”平移至离出口最近的“浅层缓冲区”。车主到达后，即可实现亚秒级的物理响应，“即取即走”。

3.5.3 边缘视觉防线与物理应急熔断时序

在物理信息系统（CPS）架构中，安全性拥有超越一切业务逻辑的最高优先级。对于交接区儿童误入、异物坠落等极端工况，依赖“摄像头->云端网络->云端识别->指令下发”的传统长链路，极易因网络抖动导致安全事故。本系统采用边缘计算极速短路设计。



核心流转逻辑解析：

边缘端自闭环推理：部署在场站本地环境的摄像机将视频流推给本地局域网内的边缘计算节点（Edge Node）。边缘节点运行轻量级的目标检测网络（如 YOLO v8），以 30 帧/

秒 以上的速率进行推流分析。

绕过云端，直接熔断 (Short-circuiting the Cloud): 一旦边缘 AI 捕获到作业红线内存在非许可生命体或异物入侵，它将绝对禁止先向云端请求决策。边缘节点将直接通过工业局域网（如 Modbus/gRPC 协议），向本地硬件控制柜（PLC）下发底层电气急停指令，确保物理隔离栏落下、AGV 瞬间断电锁死。动作闭环控制在数十毫秒以内。

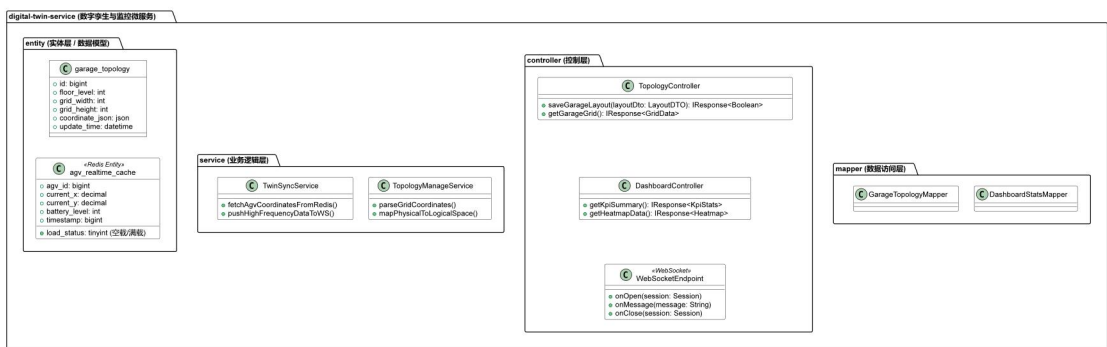
异步上报与大屏阻断：在物理机器安全停转后，边缘节点才会向中心云层发送最高优先级的 Emergency Alert 异步告警。监控大屏（WebSocket）接到告警后，立即强制弹出红色监控流画面，伴随强蜂鸣警报，等待人工介入处理与系统软复位。

3.6 数字孪生与监控微服务

数字孪生与监控微服务是 ParkVision 系统实现“所见即所得”监管逻辑的核心组件。它通过构建物理场站与数字空间的 1:1 高保真镜像同步，将复杂的分布式计算与隐蔽的机械运动透明化地呈现给运维人员，并作为异常告警处置与人工应急接管的唯一核心入口。

3.6.1 组件内部结构与高频同步架构

该微服务摒弃了传统的“请求-响应”模式，转而构建基于 WebSocket 全双工通信的实时流式架构，确保物理世界的每一厘米位移都能在数字模型中即时反馈。



双轴同步逻辑：

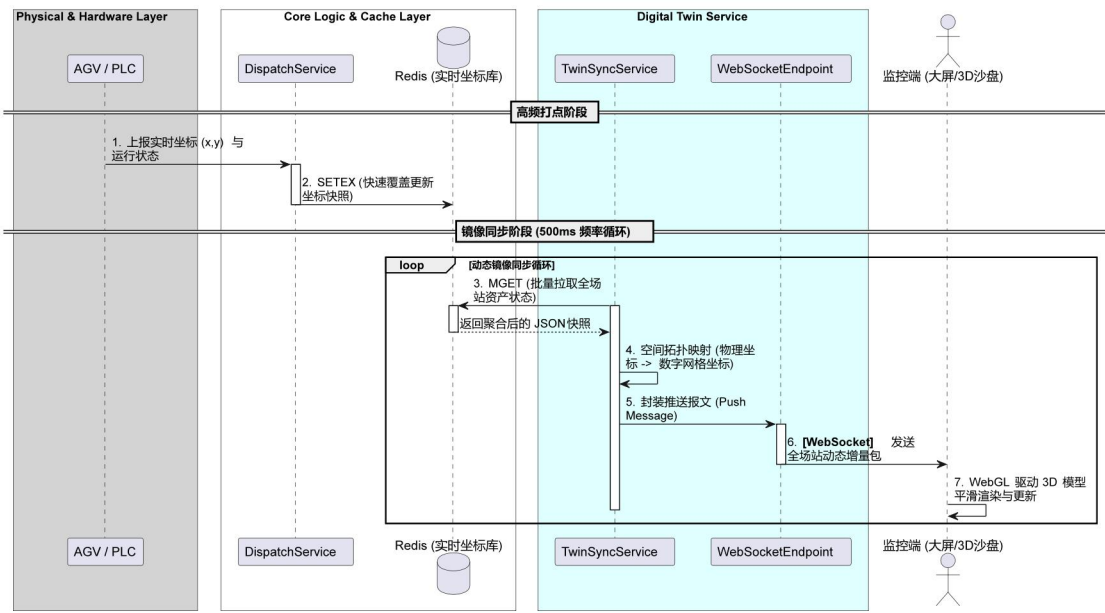
空间拓扑同步 (TopologyManageService): 负责维护静态资产镜像。将物理库位的坐标 (x, y, z)、充电桩状态映射为前端 WebGL 渲染所需的网格 (Mesh) 元数据。

高频状态映射 (TwinSyncService): 专门处理动态资产。它高频轮询 Redis 中的 AGV 坐标缓存，并将其封装为轻量级的 JSON 数据包，通过 WebSocketEndpoint 主动推送到监控端。

多维监控仪表盘 (DashboardController): 汇总全系统的 KPI 指标，包括场站实时周转率、热力图分布以及基于 AI 预测的流量趋势，为监管人员提供宏观决策支持。

3.6.2 库位状态与 AGV 高频坐标同步时序

为了保障 3D 数字沙盘的丝滑动效，系统必须在毫秒级时延内完成“感知-传输-渲染”的链路。



核心流转逻辑解析：

状态变化打点：当底层 AGV 或库位状态发生任何细微变动（由 dispatch-service 触发），信息首先写入 Redis 极速缓存。

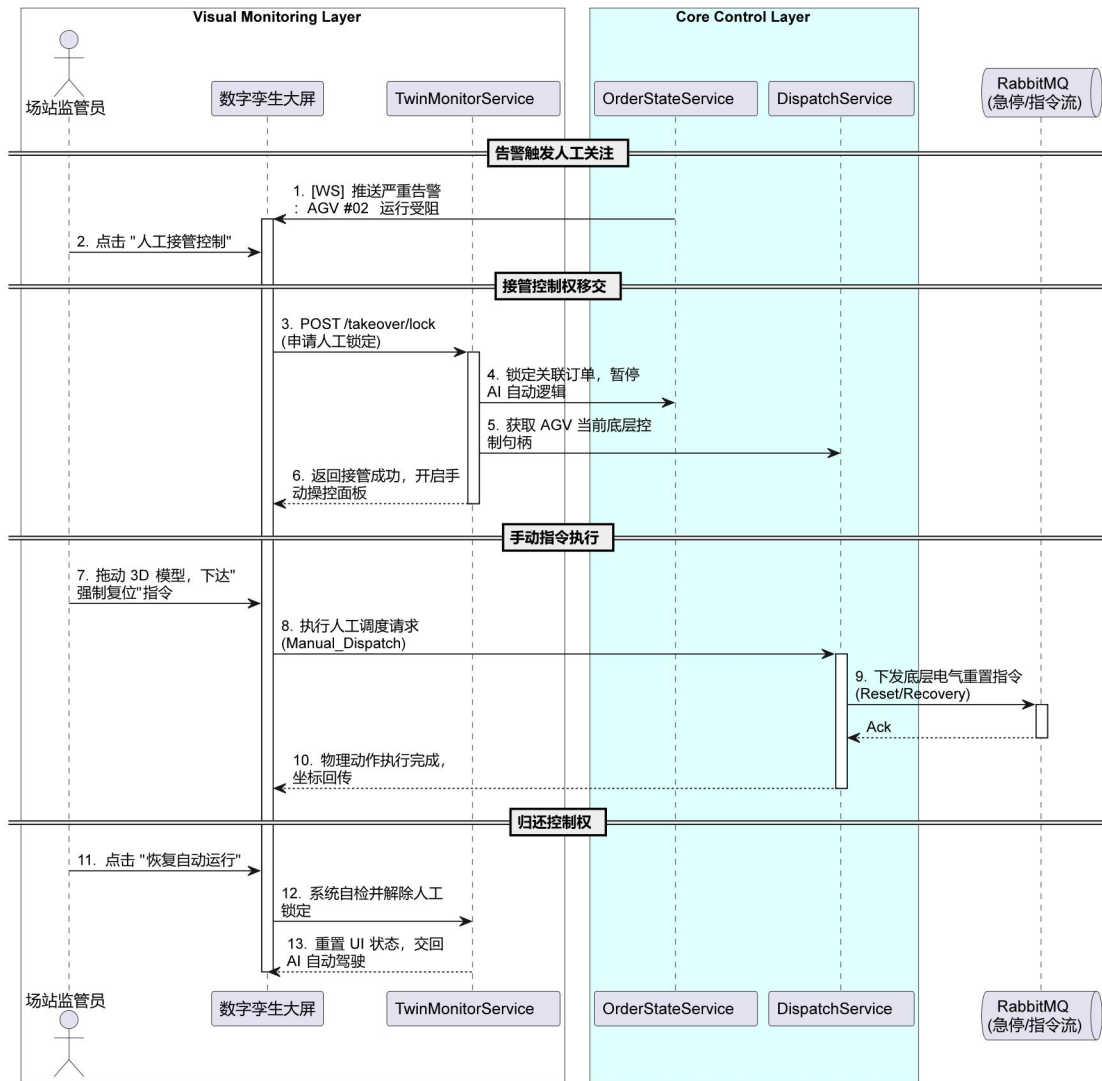
高速轮询与聚合：TwinSyncService 定时器以每 500ms 一次的频率从 Redis 提取全场站的最新快照数据。

长连接主动推送：服务端不再等待前端刷新，而是通过已建立的 WebSocket 长连接，将包含所有 AGV 当前坐标及库位占用情况的增量包推送至前端。

WebGL 动态渲染：监控端（大屏）接收报文后，驱动 WebGL/Canvas 引擎重绘 3D 模型。通过线性插值算法，在视觉上实现 AGV 在数字网格中平滑移动的效果，达成 1:1 物理同步。

3.6.3 异常干预与人工接管时序

数字孪生不仅仅是视觉展示，更是应急管理的“遥控器”。当 AI 决策中心发现系统逻辑无法处理的极端工况时，监控人员可以通过大屏实现人工介入。



核心流转逻辑解析：

告警透传：边缘 AI 捕获到物理告警（如上一章提到的异物入侵），通过双向 WebSocket 强行在监控端弹出高危弹窗。

人工锁定：监管人员在大屏点击“人工接管”按钮。数字孪生服务立即向 order-service 发起请求，锁定当前所有受影响的订单状态，禁止 AI 自动调度介入。

指令直达物理层：运维人员通过 3D 界面下达手动复位或强制移位指令。该请求绕过常规业务逻辑，通过调度引擎直接转换为底层机械动作，引导 AGV 脱离险境。

日志闭环与交回：所有手动操作过程被完整记录（留痕审计），操作完成后系统执行自检，确认安全后将控制权交还给 AI 自动逻辑。

4. 接口设计

接口设计是 ParkVision 系统实现软硬件协同与业务闭环的技术契约。本章将详细说明系统与外部第三方服务的集成接口，以及内部微服务之间基于 RESTful 架构的通信规范。

4.1 外部集成接口

为了构建完整的智慧停车生态，ParkVision 系统集成了支付、导航、生成式 AI 及工业控制等多个维度的外部接口。

接口分类	接口名称	描述	核心参数
支付服务	微信支付统一下单 (V3)	生成预支付订单，车主在小程序内完成停车费、加急费的组合支付。	mchid, out_trade_no, amount
地图导航	高德/百度 LBS API	提供从城市主干道到场站交接区的路径规划及室内外一体化导航。	origin, destination, key
AI 大模型	LLM 语义处理接口	对接 ChatGPT/通义千问，实现意图解析（NLP）与智能报表生成（Text-to-SQL）。	prompt, context, temperature
硬件网关	gRPC 控制指令集	与底层 AGV 控制器（PLC）通信，下发机械寻路路径及接收实时坐标。	agv_id, route_nodes, speed

4.1.1 支付服务接口：微信支付统一下单

接口名称：微信支付统一下单

交互协议：HTTPS / XML (微信支付标准)

核心参数：mchid (商户号), out_trade_no (商户订单号), total_fee (分), notify_url (回调地址)

业务逻辑：用于在 C 端小程序中生成预支付订单。系统将停车费、加急费、充电费进行组合计算后，向微信平台发起请求，获取预支付会话标识，以实现安全的商业变现闭环。

4.1.2 地图导航接口：高德 LBS API

接口名称：高德路径规划与室内外导航 API

交互协议： HTTPS REST API

核心参数： origin (起点经纬度), destination (场站交接区坐标), key (应用密钥)

业务逻辑： 解决用户“入库前”的导流问题。系统根据车辆当前位置，提供从城市主干道到具体场站入口的路径规划，并在进入场站后提供室内引导说明，确保用户精准到达 AGV 交接区。

4.1.3 AI 大模型接口：LLM 语义处理接口

接口名称： ChatGPT / 通义千问 语义透传接口

交互协议： HTTPS / JSON / 流式传输 (SSE)

核心参数： prompt (提示词), context (业务上下文), temperature (随机性控制)

业务逻辑： 系统的“大脑”外脑。通过调用大模型 API，系统能够将用户的模糊语音转化为结构化指令，并将 B 端管理者的运营指标需求自动转化为可执行的 SQL 脚本（Text-to-SQL），极大提升了人机交互的智能化水平。

4.1.4 硬件网关接口：gRPC 控制指令集

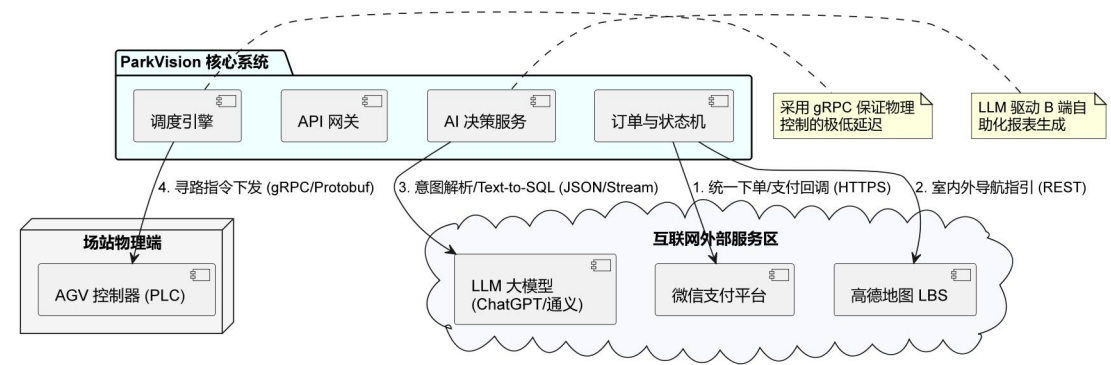
接口名称： 底层 AGV 控制器 (PLC) 通信接口

交互协议： gRPC (基于 HTTP/2)

核心参数： agv_id (设备唯一号), route_nodes (路径坐标点数组), speed (额定速度)

业务逻辑： 物理执行的终点。系统采用高性能的 gRPC 协议与本地 PLC 硬件通信，主要考虑其具备的双向流通信能力与极低的反序列化开销，确保寻路路径点位能够以毫秒级时延精准下达。

4.1.5 系统外部交互全景图



4.2 内部微服务 RESTful API

ParkVision 内部各微服务之间遵循 RESTful 架构风格 进行同步通信。为了保障分布式环境下的调用稳定性，系统统一采用 JSON 作为数据交换格式，并制定了严谨的全局响应规范。

4.2.1 接口协议与通用标准

所有内部接口均通过 Spring Cloud Gateway 进行权限透传，并遵循以下开发契约：

通信协议： HTTP/1.1

统一返回体结构：

为简化前端逻辑，所有接口均返回封装好的 ApiResponse<T> 实体，包含：

success (Boolean): 业务处理是否成功。

code (Integer): 全局状态码（如 200-成功，401-鉴权失败，500-业务异常）。

message (String): 异常描述信息。

data (T): 具体的业务载体数据。

4.2.2 核心业务接口列表

系统针对停车、调度、AI 预测等核心场景，定义了如下关键 API 路径。

调用模块	接口路径 (Path)	请求方法	核心职责说明
订 单 服务	/api/v1/orders/park	POST	车主确认到场后调用，触发库位锁定并启动状态机。
订 单 服务	/api/v1/orders/suspend	POST	执行 Touch-and-Go 逻辑，挂起当前订单并驱动车辆移至交接区。
AI 服 务	/api/v1/ai/intent/parse	POST	拦截用户自然语言输入，解析并提取出“取物”、“挪车”等结构化需求。

订 单 服务	/api/v1/dispatch/priority	PATCH	动态修改调度任务权重，实现 VIP 抢占出库的优先级提升。
AI 服 务	/api/v1/ai/forecast/peak	GET	获取未来 2 小时的预测流量分值，作为预调度策略的触发开关。

4.2.3 接口实现范例与报文结构

本节展示系统在 Spring Boot 框架下的具体 API 实现逻辑。

```
@RestController
@RequestMapping("/api/v1")
@Validated
public class ParkVisionApiDraft {

    @PostMapping("/auth/login")
    public ApiResponse<LoginResp> login(@Valid @RequestBody LoginReq req) {
        return ApiResponse.ok(new LoginResp(
            accessToken: "mock-token-owner-001",
            role: "owner",
            userId: "u_10001",
            LocalDateTime.now().plusHours(hours: 8)
        ));
    }

    @PostMapping("/owner/vehicles")
    public ApiResponse<VehicleResp> bindVehicle(@Valid @RequestBody BindVehicleReq req) {
        VehicleResp resp = new VehicleResp();
        resp.setVehicleId(vehicleId: "v_10001");
        resp.setPlateNo(req.getPlateNo());
        resp.setEnergyType(req.getEnergyType());
        resp.setDefaultVehicle(req.isDefaultVehicle());
        return ApiResponse.ok(resp);
    }

    @GetMapping("/owner/vehicles")
    public ApiResponse<List<VehicleResp>> listVehicles(@RequestParam("ownerId") String ownerId) {
        VehicleResp v1 = new VehicleResp();
        v1.setVehicleId(vehicleId: "v_10001");
        v1.setPlateNo(plateNo: "京A12345");
        v1.setEnergyType(energyType: "EV");
        v1.setDefaultVehicle(defaultVehicle: true);
        return ApiResponse.ok(List.of(v1));
    }
}
```

代码实践重点：

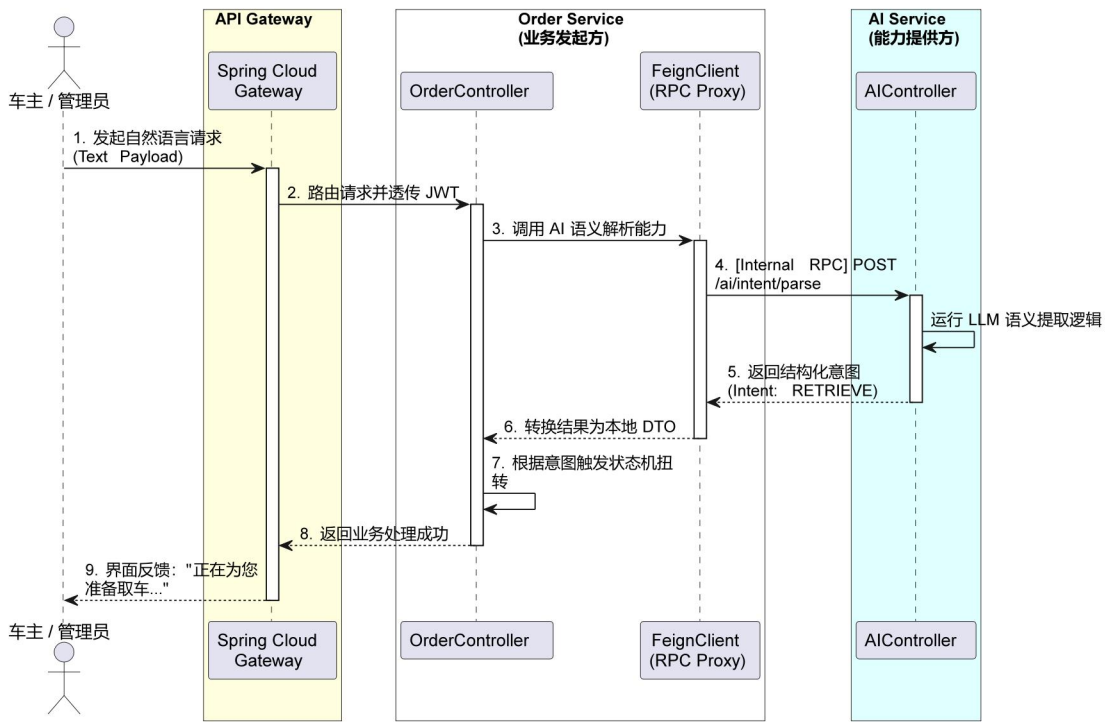
分层鉴权：通过 `@PostMapping("/auth/login")` 实现基于 JWT 的安全准入。

DTO 数据传输：使用 `@RequestBody @Valid BindVehicleReq` 模式，确保入参在进入业务层前已完成格式校验（JSR-303）。

Restful 命名规范：资源路径采用复数形式（如 `/owner/vehicles`），并配合 POST/GET 方法实现资源的增删改查。

4.2.4 内部跨服务 RPC 调用时序图

在实际业务中，一个前端请求往往需要横跨多个微服务协作。以下时序图展示了“用户通过自然语言发起请求”时，内部微服务是如何通过 OpenFeign 进行级联调用的。



核心逻辑解析：

意图初审：订单服务接收到用户的文字输入（如“我想现在取车”）。

远程语义解析：订单服务通过内网 RPC 向 AI 服务发起 /intent/parse 请求。AI 服务完成解析后返回结构化的 IntentResult（Action=Retrieve, Target=Vehicle_A）。

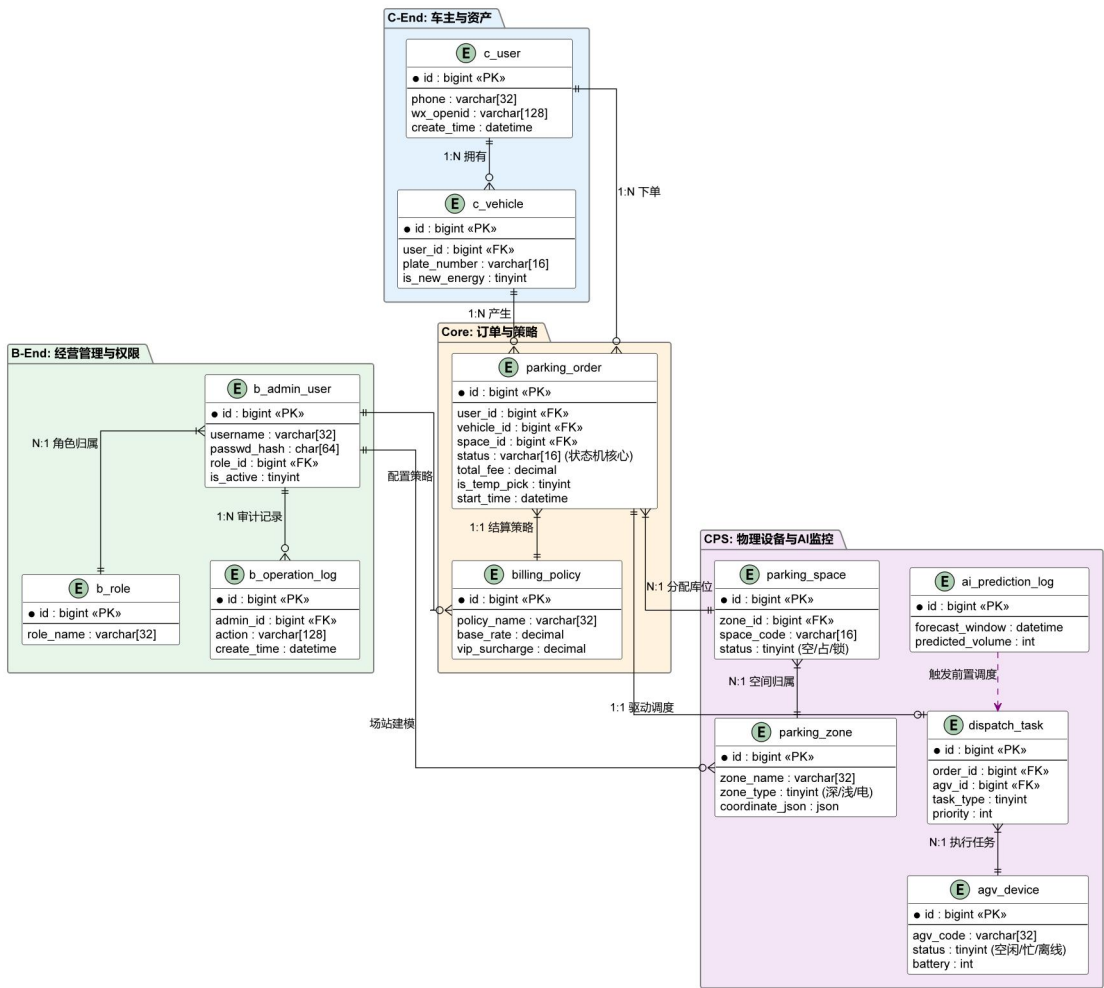
策略联动：订单服务根据解析结果，再次向调度服务发起权重调整或任务生成请求，完成跨服务的业务闭环。

5. 数据设计

数据设计是 ParkVision 系统持久化能力的保障。系统采用“关系型数据库（MySQL）驱动业务 + 缓存数据库（Redis）驱动实时状态”的双模存储架构。本章将详细描述系统的数据库模型、实体关系及核心表结构。

5.1 数据库关系图

系统的数据库设计遵循第三范式（3NF），通过逻辑分区将数据划分为 C 端用户、B 端管理、核心业务、物理监控四大领域。



核心关联逻辑：

用户-资产关联： c_user 通过 1:N 关系拥有多辆 c_vehicle。

订单-业务关联： parking_order 作为核心表，向上关联用户与车辆，向下关联具体的 parking_space（库位）与 billing_policy（计费策略）。

监控-执行关联： ai_prediction_log 产生的预判数据驱动 dispatch_task（调度任务）的生成，任务最终分配至 agv_device 执行。

5.2 C 端用户与车辆资产表

5.2.1 C 端用户表 (c_user)

属性名	介绍	数据类型	长度
id	用户身份唯一识别编号 (PK)	bigint	20
phone	车主联系手机号	varchar	32
wx_openid	微信身份识别 ID	varchar	128
nickname	微信昵称	varchar	32
account_status	账号状态 (0:正常, 1:冻结)	tinyint	1
create_time	账号注册时间	datetime	

5.2.2 车辆资产表 (c_vehicle)

属性名	介绍	数据类型	长度	说明
id	车辆唯一标识 (PK)	bigint	20	主键，系统自增
user_id	所属车主 ID (FK)	bigint	20	关联 c_user 表
plate_number	车牌号	varchar	16	唯一，如“浙 A12345”
is_new_energy	是否新能源	tinyint	1	0:否, 1:是 (影响充电位分配)
create_time	绑定时间	datetime	-	用于资产溯源

5.3 B 端经营管理与权限表

5.3.1 B 端用户表 (b_admin_user)

属性名	介绍	数据类型	长度	说明
-----	----	------	----	----

id	管理员唯一标识 (PK)	bigint	20	系统自动递增
username	登录账号名	varchar	32	唯一，不可重复
passwd_hash	密码散列值	char	64	采用 SHA-256 加盐存储
role_id	所属角色 ID (FK)	bigint	20	关联 b_role 表
phone	管理员联系电话	varchar	32	用于异常告警短信推送
is_active	账号激活状态	tinyint	1	0:停用, 1:正常
last_login	最后登录时间	datetime		用于审计安全性

5.3.2 系统角色表 (b_role)

属性名	介绍	数据类型	长度	说明
id	角色唯一 ID (PK)	bigint	20	主键
role_name	角色名称	varchar	32	如：超级管理员、场站运维员、财务监控
permissions	权限权限集合	text	-	存储权限标识符的 JSON 数组

5.4 物理场站基建表

5.4.1 库位表 (parking_space)

属性名	介绍	数据类型	长度	说明
-----	----	------	----	----

id	库位唯一 ID (PK)	bigint	20	主键，系统自动递增
space_code	物理编码	varchar	16	场站内部编号，如 A-01-102
zone_id	所属区域 ID (FK)	bigint	20	关联分区表(深库/浅区/充电区)
status	库位状态	tinyint	1	0:空闲, 1:占用, 2:预约锁定, 3:故障
current_vehicle_id	当前存放车辆 ID	bigint	20	关联 c_vehicle 表，快速检索车辆位置
coordinate_json	物理坐标数据	json	-	存储 (x,y,z) 坐标，用于数字孪生渲染

5.4.2 场站分区表 (parking_zone)

属性名	介绍	数据类型	长度	说明
id	分区唯一 ID (PK)	bigint	20	主键
zone_name	分区名称	varchar	32	如：A 区-深层存储、B 区-浅层缓冲、充电区
zone_type	分区属性	tinyint	1	1:普通, 2:充电, 3:预调度缓冲
coordinate_json	分区边界点	json	-	用于数字孪生大屏的分区框选渲染

5.5 核心业务与调度策略表

5.5.1 停车订单表 (parking_order)

属性名	介绍	数据类型	长度	说明
id	订单流水号 (PK)	bigint	20	唯一订单标识
user_id	车主 ID (FK)	bigint	20	关联 c_user 表
vehicle_id	车辆 ID (FK)	bigint	20	关联 c_vehicle 表
status	核心状态机	varchar	16	存/停妥/取物挂起/取车中/完成
space_id	分配库位 ID	bigint	20	关联 parking_space 表
is_temp_pick	临时取物标志	tinyint	1	0:否, 1:是 (触发 Touch-and-Go 逻辑)
total_fee	累计费用	decimal	10,2	包含基础+加急费+补能服务费
start_time	入库时间	datetime	-	用于阶梯计费起始点计算

5.5.2 计费策略表 (billing_policy)

属性名	介绍	数据类型	长度	说明
id	策略 ID (PK)	bigint	20	主键
policy_name	策略名称	varchar	32	如：工作日峰谷计费、VIP 加急策略
base_rate	基础费率	decimal	10,2	元/小时
vip_surcharge	VIP 附加费	decimal	10,2	用于加急插队的一次性费用

5.5.3 调度任务表 (dispatch_task)

属性名	介绍	数据类型	长度	说明
-----	----	------	----	----

		型	度	
id	任务 ID (PK)	bigint	20	唯一调度流水号
order_id	关联订单 ID (FK)	bigint	20	关联 parking_order
agv_id	执行 AGV ID (FK)	bigint	20	指定执行的机器人硬件
task_type	任务类型	tinyint	1	1:存车, 2:取车, 3:移库(预调度)
priority	实时权重分值	int	11	基于 Redis ZSET 维护的动态优先级
status	执行状态	tinyint	1	0:等待执行, 1:移动中, 2:已完成, 3:故障异常

5.5.4 AI 预测日志表 (ai_prediction_log)

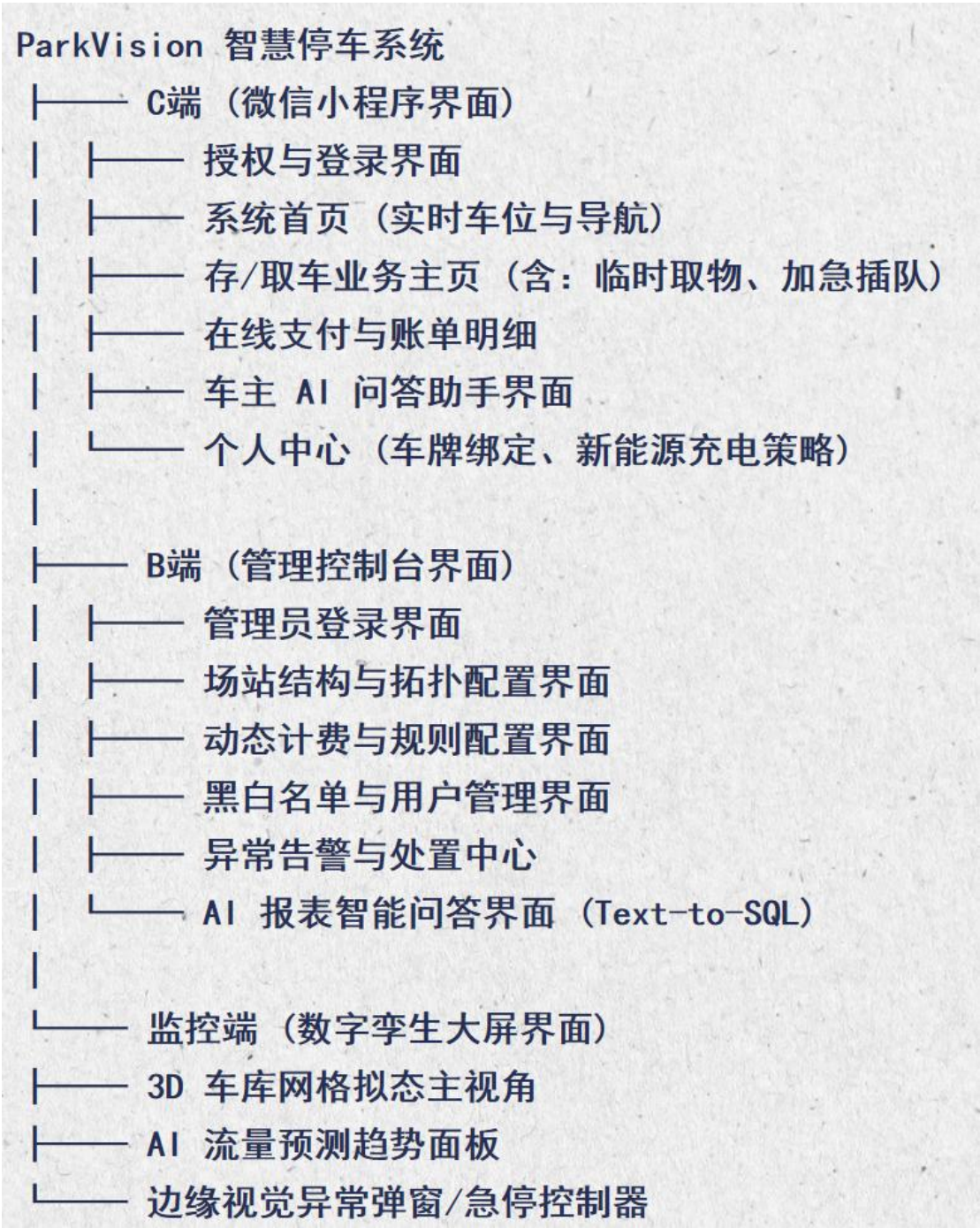
属性名	介绍	数据类型	长度	说明
id	日志 ID (PK)	bigint	20	主键
forecast_window	预测时间窗	datetime	-	预测该时刻的流量趋势
predicted_volume	预测出库量	int	11	AI 算法推演出的数值
actual_volume	实际出库量	int	11	用于后期 AI 模型的准确率自校准

6. 界面与人机交互设计

界面设计是 ParkVision 系统功能的最终呈现载体。本系统针对不同角色的业务痛点，构建了“三端协同”的交互体系：面向车主的移动化服务端、面向管理者的数智化控制台，以及面向监管者的数字孪生大屏。

6.1 设计原则与界面导航图

系统的 UI 设计遵循“业务透明化、交互语义化、监控实时化”的原则。通过界面导航图，可以清晰地看到系统的功能分布。



C 端 (微信小程序)：侧重于极致的流程简化，将底层复杂的 AGV 调度封装为“存、取、缴、问”四个核心动作。

B 端 (管理控制台)：侧重于数据广度与管理深度，通过 AI 赋能的自然语言报表降低管

理者的技术门槛。

监控端 (数字孪生大屏)：侧重于全局态势感知，利用 3D 渲染技术消除物理世界的“视觉死角”。

6.2 C 端车主服务界面设计

C 端界面旨在解决车主在存取车过程中的焦虑感,提供全流程的引导与灵活的异常处理。

6.2.1 授权登录与无感化主页



设计逻辑：采用单键快捷登录。页面保持高度留白，突出核心操作入口。主页动态关联车主名下的活跃订单，实现“一键触达”。

6.2.2 存取业务与“Touch-and-Go”交互



核心交互：

状态看板： 实时展示车辆位置（如“B2 层 深库区 - A12 位”）及当前停放时长。

业务挂起弹窗： 当用户点击“临时取物”时，界面立即切换为倒计时拟态圆环（如图右侧所示）。这种高饱和度的动态视觉反馈，能有效缓解用户等待车辆运出时的枯燥感，并明确操作时限。

加急申请： 界面底部显著位置提供“VIP 加急取车”入口，并预估可节省的时间（如“预计省时 8 分钟”），引导用户完成价值溢价交互。

6.2.3 AI 智能助手交互界面



设计逻辑：采用对话式 UI (CUI)。AI 助手不仅能回答静态问答，更能实时接入后台预测引擎。例如当用户询问“排队提车要多久？”时，AI 能够通过分析当前队列权重，给出精准的“12-15 分钟”预估值，并主动推销“VIP 加急调度”功能，实现业务逻辑的主动营销。

6.3 B 端管理控制台界面设计

B 端界面设计核心在于“去专业化”，使非技术背景的管理者也能轻松操控复杂的物理场站。

6.3.1 管理员安全登录



设计逻辑：采用分栏式布局，左侧宣传系统核心价值（下一代智慧停车全流程管理平台），右侧提供严格的权限校验入口。

6.3.2 AI 智能报表与 Text-to-SQL 交互



核心交互：

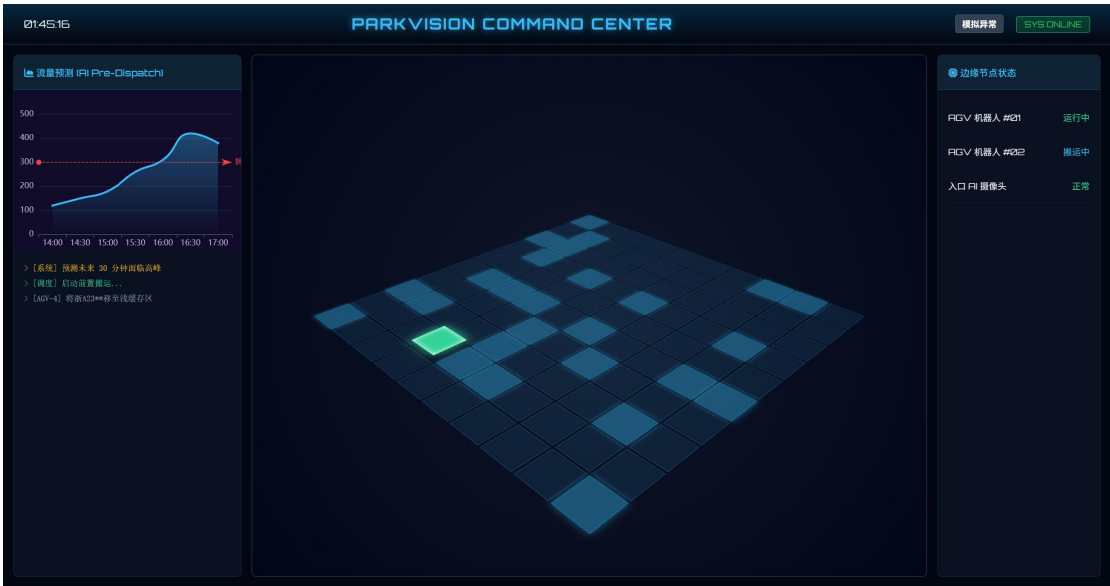
自然语言查询框：管理员无需编写 SQL 指令，只需输入“对比过去七天 VIP 加急服务带来的额外收益趋势”。

自动图表生成：系统自动编译 SQL 并返回可视化的折线图。这种“对话即报表”的交互

范式，极大地提升了运营分析的实时性，使数据真正驱动经营策略决策（如动态调整 VIP 费率）。

6.4 数字孪生监控大屏界面设计

监控大屏是系统的“终极上帝视角”，负责对场站进行 1:1 的实时数字化重映射。



3D 空间网格拟态：屏幕中央采用 WebGL 渲染的网格阵列，代表物理车库的立体库位。通过色彩明度变化，直观展示库位占用率（如亮绿色代表正在变动的库位，深蓝色代表已停放）。

AI 预测趋势面板：左上角展示流量预测曲线。当预测曲线（白线）即将穿透拥堵红线时，界面会产生预警动效，辅助人员提前开启预调度。

边缘节点状态监控：右侧实时列出 AGV 机器人的健康度与运行速度，一旦发生“异物闯入”，中央区域会立即弹出高亮的红框视频流，强行中断当前所有视觉，确保安全第一优先级。

7. 非功能性与系统保障设计

为确保 ParkVision 系统在面对高并发业务流量及极端物理环境时具备卓越的可靠性与安全性，本系统在设计阶段便引入了多层保障机制，涵盖了性能优化、分布式一致性、体系安全及可观测性四大维度。

7.1 高并发与性能优化设计

根据《需求分析文档》定义的性能指标（车主关键操作响应时间 < 2s），系统在软件层面采用了多级缓存与异步化策略。

高性能排队策略：

系统调度引擎采用 Redis ZSET 实现动态任务队列。相较于传统关系型数据库的 ORDER BY 操作，ZSET 在内存中完成 $O(\log N)$ 级别的权重重排，确保即便在数千台车辆同时排队的极端场景下，VIP 抢占与任务分发依然能够实现毫秒级响应。

软硬件异步解耦：

鉴于物理机械搬运（AGV 动作）通常耗时 3-5 分钟，系统通过 RabbitMQ/Kafka 实现请求的“削峰填谷”。前端业务请求在完成状态机扭转后即刻返回成功，后续物理搬运流程转入异步执行，防止长耗时的机械动作阻塞系统线程池，极大提升了系统的整体吞吐量。

多级缓存架构：

系统对高频访问数据（如库位状态、AGV 实时坐标）执行“近数策略”。动态坐标数据优先在边缘端聚合，通过 WebSocket 增量推送到监控端，同时在云端 Redis 中维持一份热点镜像，降低对 MySQL 核心数据库的并发读取压力。

7.2 系统高可用与可靠性保障

针对物理信息系统中常见的网络波动与硬件失联风险，本系统设计了严密的容错与对齐机制。

分布式一致性与防死锁：

在跨微服务的订单流转中，系统严格执行 Redis Redlock 分布式锁协议。在存取车操作发起前，强制锁定 vehicle_id 与 space_id。若发生网络超时或获取锁失败，系统将自动执行事务回滚（Rollback）与状态对齐，确保“数字状态”与“物理位置”绝对一致，从底层杜绝了死锁与车辆碰撞风险。

重试机制与幂等性设计：

针对微信支付回调、硬件指令下发等关键环节，系统设计了具备**指数退避算法（Exponential Backoff）**的自动重试机制。同时，通过全局唯一请求 ID（TraceID）保证接口幂等性，确保在网络抖动导致重复推送时，不会引发账单重复核销或 AGV 重复动作。

数字孪生状态纠偏：

系统在数字孪生模块内置了“心跳监测”与“定期全量对齐”逻辑。每隔 60 秒，系统会自动比对物理层 PLC 反馈的坐标与数据库记录的库位状态，一旦发现偏差，立即以物理层数据为准进行数字侧强制修正（Anti-drift），保障监控系统的真实性。

7.3 体系安全设计

安全设计涵盖逻辑安全、数据安全与物理安全三个维度。

访问控制安全：

系统网关层严格执行 JWT 无状态鉴权。结合 RBAC 权限模型，任何非法越权（如 C 端车主尝试调用 B 端配置接口）请求均会在最外层被直接熔断。

敏感数据保护：

密码加密：管理员与用户敏感信息采用 SHA-256 加盐散列存储，确保存储介质即便泄露，原始密码也无法被反向推算。

支付脱敏：所有的资金结算报文均符合微信支付安全规范，敏感字段在日志记录中执行自动遮蔽（Masking）处理。

物理安全（边缘急停机制）：

这是本系统的核心安全特色。系统在边缘计算节点实现了物理防护指令的“极速短路”。当检测到生命体闯入时，边缘节点绕过云端业务层，直接下发硬件急停信号。该设计将安全防御的时延由秒级缩短至毫秒级，守住了人机共存环境下的安全红线。

7.4 可观测性与异常追溯设计

为了支撑后期的系统维护与快速故障排查，系统构建了全链路的可观测性方案。

全局链路追踪：

通过集成 SkyWalking 或 Zipkin 追踪工具，为每一个用户请求分配唯一的 TraceID。无论是前端报错还是硬件卡顿，运维人员均可一键追溯该请求横跨网关、微服务、MQ、直至底层 PLC 的全路径执行细节与耗时。

智能告警分级：

系统建立了分级告警矩阵。

普通异常（L1）：如支付超时、车位分配失败，通过即时通讯工具通知运营人员。

高危告警（L2）：如 AGV 掉线、PLC 报错，立即触发监控大屏强弹窗并短信通知运维主管。

灾难响应（L3）：如火灾监控触发或异物入侵，系统自动执行全场站物理锁定并启动应急接管时序。